剑指 Web3 Offer

Web3 / Solidity / DeFi / L2 / Account Abstraction

100 道真题级面试题 · 从原理到系统设计

EVM | SECURITY | DeFi / L2 | SYSTEM DESIGN

专业面试复习手册 | 2026-09 技术口径

适用岗位：Smart Contract Engineer · Protocol Engineer · Web3 Backend · Wallet / AA · Security · DeFi

编写说明

本书采用问题驱动的面试学习结构：先给出面试场景下的短回答，再展开底层原理、工程边界、追问和常见误区。组织思路参考《剑指 Offer》所代表的“以问题带知识点”学习范式，但本书不复制其原文、题目、图表或版式细节。

“真题级”表示题目来自 Web3 岗位公开面试模式中反复出现的能力类型，并结合当前主流协议栈构造为可直接用于真实面试的问法；不把未经公开证实的问题伪装成某家公司的特定原题。

技术口径更新至 2026 年 9 月。尤其纳入 EIP-1153 Transient Storage、EIP-4844 blobs、ERC-4337 与 EIP-7702 的组合关系，以及 Solana account/program/PDA/CPI 执行模型。

使用建议 第一次阅读先掌握每题的“30 秒回答 + 记忆锚点”；第二轮补齐“深入解析 + 高频追问”；冲刺轮只刷五星高频题和系统设计题。

面试评分模型

层级	面试表现	面试官判断	目标
L1 定义	能解释名词	基础了解	不丢基础分
L2 原理	能推导为什么	真正理解	通过普通工程面
L3 Failure Mode	主动说边界/攻击面	有生产经验	高级工程师
L4 Invariant/System	能抽象状态机与信任边界	Protocol/Staff 水平	系统设计与终面

Web3 面试能力地图

底层	Blockchain / EVM	状态、交易、Gas、Storage、ABI
合约	Solidity / Security	权限、重入、Proxy、Invariant
协议	DeFi / L2 / Bridge	AMM、Lending、DA、Proof、Sequencer
账户	Wallet / AA / Multi-chain	签名、Replay、4337、7702、PDA
系统	RPC / Indexer / Ledger	Reorg、幂等、账本、可靠性、调试

一个可靠的高级回答通常遵循：定义 -> 原理 -> Failure Mode -> Mitigation -> Invariant / Trade-off。

目录

01 Blockchain 基础与密码学直觉 Q01-Q10

从“为什么需要链”开始，而不是从项目名开始。面试官关注你是否理解信任模型、状态复制、最终性和密码学承诺。

02 Ethereum / EVM：从交易到状态转换 Q11-Q20

本章是 Solidity 岗位的地基：你需要能从 calldata、opcode、storage slot 一直追到整笔交易状态转换。

03 Solidity 工程：语言语义、ABI 与 Gas Q21-Q30

这一章不考“会不会写语法”，而是考语言语义在 EVM 上的成本、安全后果与接口边界。

04 智能合约安全：从漏洞列表到 Invariant Q31-Q40

真正的安全面试不是背 SWC 编号，而是识别资产、信任边界、状态不变量、外部调用和经济假设。

05 DeFi：AMM、Lending 与经济不变量 Q41-Q50

DeFi 面试看的是“公式如何成为资金状态机”。你需要从定价曲线、份额 accounting、清算与 oracle 风险推导系统行为。

06 Layer2、Rollup、ZK 与 Bridge Q51-Q60

扩容题要按 execution、data availability、settlement、proof、sequencing 五层拆解。只说“L2 更快更便宜”远远不够。

07 钱包、签名与 Account Abstraction Q61-Q70

钱包题的本质是授权系统设计：谁能签、签什么、签名能否重放、谁付 gas、账户如何恢复和限制权限。

08 Oracle、MEV 与 Web3 基础设施 Q71-Q80

合约不是孤岛。真实系统依赖 oracle、mempool、RPC、node、indexer 和数据库；基础设施错误同样会导致资金与账本事故。

09 Solana 与 Multi-chain：另一种执行模型 Q81-Q90

不要把 Solana 当“更快的 EVM”。它的 program/account 分离、显式账户列表、PDA/CPI 和并行执行模型带来完全不同的工程边界。

10 系统设计与现场实战 Q91-Q100

高级 Web3 面试最终会回到软件工程：实现、审查、调试、可恢复性、幂等性、账本、一致性和安全边界。

CHAPTER 01

Blockchain 基础与密码学直觉

本章目标 从“为什么需要链”开始，而不是从项目名开始。面试官关注你是否理解信任模型、状态复制、最终性和密码学承诺。

本章题目

Q01 Blockchain 到底解决了什么问题?
Q02 UTXO 模型与 Account Model 有什么区别?
Q03 Hash 在 Blockchain 中有哪些作用?
Q04 Merkle Tree 为什么特别适合 Blockchain?
Q05 什么是区块链 Finality?
Q06 PoW 与 PoS 的本质差异是什么?
Q07 Fork、Fork Choice 与 Hard Fork 有什么区别?
Q08 为什么区块链需要 Gas?
Q09 Private Key、Public Key、Address、Signature 的关系是什么?
Q10 为什么“Code is Law”并不完全成立?

Q01 Blockchain 到底解决了什么问题?

难度 ★★ | 高频度 ★★★★★ | 岗位 全岗位

面试官意图 区分“分布式数据库”与“在缺少统一可信写入方时维护公共状态机”的本质差异。

30 秒回答

Blockchain 解决的核心不是“把数据库复制很多份”，而是在互不完全信任的参与者之间，通过共识、密码学承诺和经济约束，让大家对同一状态转换历史达成可验证的一致。

深入解析

传统数据库可以高效复制数据，但写权限和最终真相仍由某个组织决定。公链把“谁能提出状态变化、别人如何验证、冲突如何选择、作恶代价是什么”写进协议。

可以把链抽象为 replicated state machine：所有诚实节点从同一已确认状态出发，对同一有序交易序列执行确定性状态转换，得到同一新状态。密码学负责身份与完整性，共识负责排序与最终性，激励机制负责提高攻击成本。

必须说出的关键点

- replicated state machine
- permissionless verification
- consensus + cryptography + incentives
- append-only history 只是结果，不是完整定义

高频追问

- 为什么 PostgreSQL 多副本不能替代 Ethereum？答：它没有解决无统一管理员时的写权限与冲突裁决。
- 联盟链和公链的信任假设有何不同?
- 如果所有参与者彼此信任，还需要区块链吗？通常不需要。

常见误区 / Gotchas

- 把“去中心化”当作唯一价值；没有解释状态机与信任模型。
- 把“不可篡改”说成绝对属性；现实中仍取决于共识安全与社会层。

记忆锚点 数据库解决“怎么存”，Blockchain 更难的问题是“谁有资格改变公共状态，以及别人为什么相信这个改变”。

Q02 UTXO 模型与 Account Model 有什么区别?

难度 ★★ | 高频度 ★★★★☆ | 岗位 协议/后端/钱包

面试官意图 测试你是否能从状态表示方式推导并发性、可组合性和钱包工程差异。

30 秒回答

UTXO 把资产表示成一组未花费输出，交易显式消费旧输出并产生新输出；Account Model 直接维护账户余额、nonce、code 和 storage，通过交易修改全局状态。前者依赖关系更显式，后者更适合复杂共享状态和智能合约。

深入解析

UTXO 的每个输入都明确指向之前的输出，因此冲突检测天然清晰；不同 UTXO 的交易在理论上更容易并行。Account Model 更像共享可变状态，开发体验直接，但交易之间可能有隐含状态依赖。

Ethereum 的 contract account 将 code 与 storage 绑定在同一地址语义下，DeFi 合约之间可以同步组合调用；Bitcoin 风格 UTXO 需要用输出脚本和交易图表达状态。

必须说出的关键点

- UTXO: consume-and-create
- Account: mutate shared state
- 并行性 vs 可组合性
- 钱包 coin selection 是 UTXO 特有的重要工程问题

高频追问

- 为什么 Solana 又不完全等同于 Ethereum Account Model？因为 program 与 data account 分离并显式声明账户访问。
- UTXO 是否一定比 Account Model 更安全？不是，安全取决于协议和实现。
- Account nonce 为什么重要？防重放并确定同账户交易顺序。

常见误区 / Gotchas

- 只回答“Bitcoin 用 UTXO，Ethereum 用账户”而不推导工程后果。
- 把 UTXO 理解成“余额分片”而忽略 spend condition。

记忆锚点 UTXO 把依赖写在交易输入里；Account Model 把依赖藏在共享状态读写里。

Q03 Hash 在 Blockchain 中有哪些作用?

难度 ★★ | 高频度 ★★★★☆ | 岗位 全岗位

面试官意图 考察密码学基础是否能连接到区块、交易、Merkle commitment、地址和确定性部署。

30 秒回答

Hash 是把任意长度输入映射为固定长度摘要的单向承诺工具。链上常见用途包括区块链接、交易标识、Merkle root、状态承诺、签名前消息摘要、CREATE2 地址计算和 PoW。

深入解析

面试中要区分 collision resistance、preimage resistance 和 second-preimage resistance。区块链通常不是“用 hash 加密数据”，而是用 hash 做内容承诺与完整性检测。

Merkle root 的意义尤其重要：它把大量叶子压缩成一个固定长度 commitment，随后可以用对数规模的 proof 证明某个叶子的成员关系。

必须说出的关键点

- 不要说“hash 可逆解密”
- Keccak-256 与 SHA-256 不是同一个算法
- commitment 的核心是变更任意输入都会改变摘要
- 地址推导与函数 selector 都依赖 hash

高频追问

- Hash collision 是否意味着可以直接偷币？通常还需要具体协议利用路径。
- 为什么不能用普通 checksum 替代密码学 hash?
- 什么是 domain separation？避免同一字节串在不同语义域被混用。

常见误区 / Gotchas

- 把 encryption、signature、hash 三者混为一谈。
- 认为 hash 碰撞“理论存在”就等同于“工程可利用”。

记忆锚点 Hash 不是加密，而是廉价、确定、难伪造的内容指纹与承诺。

Q04 Merkle Tree 为什么特别适合 Blockchain?

难度 ★★ | 高频度 ★★★★★ | 岗位 协议/后端/安全

面试官意图 看你能否解释“commitment + logarithmic proof”，而不是只会画二叉树。

30 秒回答

Merkle Tree 用叶子 hash 逐层合并得到 root。对 N 个叶子，证明某个叶子属于 root 通常只需要 O(log N) 个 sibling hashes，因此非常适合链上保存一个 root、链下提供 proof。

深入解析

Merkle proof 的验证者不需要获取全部数据，只需沿叶子到根的路径重新计算 hash。它常用于 airdrop allowlist、rollup state commitment、轻客户端证明和数据完整性检查。

工程里要特别注意叶子编码、左右顺序、重复叶子、排序策略和 domain separation，否则“数学上是 Merkle Tree”并不保证协议语义安全。

proof size ≈ O(log N)

N 为叶子数量

必须说出的关键点

- root 是集合/序列的紧凑 commitment
- membership proof O(log N)
- proof 设计与 leaf encoding 同样重要
- Merkle Patricia Trie / Sparse Merkle Tree 是不同结构

高频追问

- 如果树排序了，证明语义有什么变化?
- non-membership proof 如何实现？Sparse Merkle Tree 更自然。
- 为什么 Merkle proof 不能证明“现实世界事实是真的”？它只证明数据被某个 root 承诺。

常见误区 / Gotchas

- 把 Merkle root 说成“压缩后可还原全部数据”。
- 忽略 leaf 编码碰撞或 abi.encodePacked 的歧义风险。

记忆锚点 Merkle Tree 的价值不是省存储本身，而是用一个 root 换来便宜的可验证证明。

Q05 什么是区块链 Finality?

难度 ★★★ | 高频度 ★★★★☆ | 岗位 协议/交易所/后端

面试官意图 测试你能否区分“已打包、确认、最终化”，这对充值、桥和交易所风控非常关键。

30 秒回答

Finality 表示某个历史状态在协议安全假设下不再会被正常分叉规则回滚。PoW 更常讨论概率确认深度；PoS 系统可提供显式经济最终性。

深入解析

一个交易“出现在最新区块”并不等于业务上可以立即不可逆记账。交易所入金、跨链桥和大额结算通常根据链的 reorg 特征设置确认策略。

最终性不是单纯时间概念，而是安全假设：如果超过协议容忍阈值的验证者作恶，或者社会层发生非常规 fork，所谓“最终”仍有更高层的例外。

必须说出的关键点

- included != confirmed != finalized
- probabilistic finality vs explicit finality
- 业务确认策略取决于风险金额和链属性
- reorg handling 必须是系统能力

高频追问

- 为什么交易所对不同链需要不同 confirmation 数?
- L2 的 finality 要区分 L2 inclusion、L1 data posting、L1 finality。
- 如果已 credit 的充值被 reorg 掉怎么办？需要 ledger rollback/hold 策略。

常见误区 / Gotchas

- 把“12 个 block”当作所有链通用常数。
- 忽略应用层“soft confirmation”和协议 finality 的区别。

记忆锚点 Finality 是“历史在什么安全假设下不再回滚”，不是“等几分钟”。

Q06 PoW 与 PoS 的本质差异是什么?

难度 ★★ | 高频度 ★★★★☆ | 岗位 协议/研究

面试官意图 考察 Sybil resistance、攻击成本和最终性，不是能源口号。

30 秒回答

两者都在解决开放网络里的 Sybil resistance。PoW 把影响力绑定到可验证计算工作和能源成本；PoS 把影响力绑定到协议内可罚没的经济 stake。

深入解析

PoW 攻击者通常需要持续获得算力并承担外部资源成本；PoS 把攻击资本显式放在协议内，因此可通过 slashing、inactivity penalties 等机制增加作恶代价。

比较时应讨论 fork choice、finality、long-range attack、弱主观性、硬件门槛、资本集中和恢复机制，而不是简单说“PoS 更环保”。

必须说出的关键点

- 共同目标是 Sybil resistance
- external resource vs slashable internal capital
- PoS 通常更容易提供显式经济 finality
- 两者都有中心化压力来源

高频追问

- Nothing-at-stake 是什么历史问题?
- PoS 为什么需要 weak subjectivity checkpoint?
- 51% 在 PoW 和 PoS 下分别意味着什么?

常见误区 / Gotchas

- 把 validator 数量直接等同于去中心化程度。
- 只比较 TPS；共识机制与执行吞吐是不同层。

记忆锚点 PoW 用外部工作证明“我付出了资源”，PoS 用可罚没 stake 证明“我有经济风险暴露”。

Q07 Fork、Fork Choice 与 Hard Fork 有什么区别?

难度 ★★ | 高频度 ★★★☆☆ | 岗位 协议/后端

面试官意图 避免把短期链分叉与协议升级混为一谈。

30 秒回答

临时 fork 是多个候选链头同时存在；fork choice rule 决定节点跟随哪个候选历史；hard fork 则是协议规则发生不向后兼容的变更。三者属于不同层次。

深入解析

网络延迟或同时提议区块会产生短暂分叉，正常共识会通过 fork choice 收敛。Hard fork 是节点软件规则改变，如果旧节点不升级，它可能拒绝新规则产生的区块。

应用工程最关心的是短期 reorg：已经处理过的 logs 可能从 canonical chain 消失，因此 indexer 和交易系统不能假设“看到一次就永远存在”。

必须说出的关键点

- temporary fork 是运行时现象
- fork choice 是选择规则
- hard fork 是规则升级
- 应用必须以 blockHash/parentHash 跟踪 canonical chain

高频追问

- Soft fork 为什么可能向后兼容?
- Indexer 如何检测 reorg? 见 Q79/Q97。
- Protocol upgrade 为什么经常需要客户端协调?

常见误区 / Gotchas

- 把 hard fork 等同于“一定分裂成两条币”。
- 把“链最长”泛化到所有 PoS fork-choice。

记忆锚点 Fork 是候选历史，fork choice 是裁判规则，hard fork 是改规则。

Q08 为什么区块链需要 Gas?

难度 ★★ | 高频度 ★★★★★ | 岗位 EVM/合约/后端

面试官意图 判断你是否理解资源计量、DoS 防御和 block resource market。

30 秒回答

Gas 是确定性的计算资源计量单位。它让不同 EVM 操作按资源成本定价，限制单笔交易和单区块可执行工作量，并让用户通过费用竞争稀缺区块空间。

深入解析

如果没有 gas，一个恶意合约可以执行无限循环或大量状态写入，要求所有验证节点永久消耗资源。Gas 把计算、内存扩展、状态访问等成本显式化，并在耗尽时中止执行。

Gas 与 ETH 价格不是同一个概念：gas 表示资源量，gas price / fee 将资源量映射为支付成本。EIP-1559 又把 base fee 与 priority fee 拆开。

必须说出的关键点

- resource metering
- DoS prevention
- block resource allocation
- gas used × effective gas price 才是费用

高频追问

- 为什么 SSTORE 比 ADD 贵？因为持久状态增加长期系统成本。
- eth_call 为什么用户不付 gas？RPC 模拟仍执行 gas 规则，只是不产生链上付费。
- out-of-gas 后 state 如何处理？当前调用帧的状态变化回滚。

常见误区 / Gotchas

- 把 gas 当作矿工“随便收的手续费”。
- 认为 view/pure 在链上调用永远免费。

记忆锚点 Gas 是 EVM 的“资源预算”，fee 才是你为这份预算实际付的钱。

Q09 Private Key、Public Key、Address、Signature 的关系是什么?

难度 ★★ | 高频度 ★★★★★ | 岗位 钱包/全岗位

面试官意图 这是钱包、签名、permit、AA 的基础。

30 秒回答

私钥是秘密标量，公钥由椭圆曲线标量乘法派生；Ethereum 地址由公钥哈希截取得到。签名证明“持有私钥的人同意某个消息”，验证不需要暴露私钥。

深入解析

不要用“私钥加密、公钥解密”解释数字签名。签名与加密是不同密码学目标。Ethereum 常见 secp256k1 ECDSA 签名可以通过 ecrecover 从消息摘要和签名恢复地址。

真正安全的签名协议还必须绑定 domain、chain id、contract、nonce、deadline 等上下文，否则“签名是真的”仍可能被重放在错误场景。

必须说出的关键点

- private key 永不发送给链
- address 不是 public key 本身
- signature = authorization evidence
- replay protection 由消息设计决定

高频追问

- 助记词与私钥是什么关系？助记词通过确定性派生路径生成一组 keys。
- 为什么同一个私钥可跨链得到相同 EVM 地址？派生算法与地址规则相同。
- 硬件钱包保护了什么？尽量让私钥不离开安全设备。

常见误区 / Gotchas

- 把签名说成“把消息用私钥加密”。
- 忽略用户签的是原始文本还是经过 prefix/domain 的 digest。

记忆锚点 签名回答“谁授权了这条消息”，不是“谁加密了这段内容”。

Q10 为什么“Code is Law”并不完全成立?

难度 ★★★ | 高频度 ★★★☆☆ | 岗位 协议/产品/安全

面试官意图 看候选人是否具备现实协议治理视角。

30 秒回答

智能合约确实可以自动执行规则，但真实 Web3 系统还依赖治理、升级权限、oracle、前端、RPC、multisig、法律和社会共识，因此代码只是系统规则的一层。

深入解析

一个“immutable contract”也可能依赖可升级 oracle 或中心化 frontend；一个“去中心化协议”也可能保留 pause guardian、parameter admin 或 upgrade timelock。面试时应主动画出完整 trust boundary。

“规则写进代码”提升了可验证性，但无法消灭 specification bug、经济机制漏洞、密钥风险和社会层应急决策。

必须说出的关键点

- 区分 protocol logic 与 governance
- 列出 admin/oracle/frontend/bridge 等外部信任
- immutability 是属性，不自动等于安全
- trust minimization 比口号式 trustless 更准确

高频追问

- 如何评估一个协议真实 decentralization？列出所有可单方面改变用户资产结果的主体。
- Timelock 能解决什么？给用户观察和退出时间。
- Emergency pause 是中心化还是风险控制？取决于权限和退出设计。

常见误区 / Gotchas

- 认为部署到链上就“任何人都无法控制”。
- 忽略社会层 fork 或治理升级。

记忆锚点 审计 Web3 系统时，不只审代码，还要审“谁能改变代码依赖的世界”。

CHAPTER 02

Ethereum / EVM：从交易到状态转换

本章目标 本章是 Solidity 岗位的地基：你需要能从 calldata、opcode、storage slot 一直追到整笔交易状态转换。

本章题目

Q11 EVM 是什么?
Q12 Stack / Memory / Storage / Transient Storage 有什么区别?
Q13 Solidity 的 storage、memory、calldata 有什么区别?
Q14 CALL 与 DELEGATECALL 有什么区别?
Q15 ABI Function Selector 和 calldata 如何编码?
Q16 Mapping 在 EVM Storage 中如何定位?
Q17 CREATE 与 CREATE2 有什么区别?
Q18 EIP-1559 的费用机制如何工作?
Q19 EIP-1153 Transient Storage 解决了什么?
Q20 一笔 Ethereum Transaction 从钱包到 Finality 经历什么?

Q11 EVM 是什么?

难度 ★★ | 高频度 ★★★★★ | 岗位 Solidity/协议/安全

面试官意图 确认你是否把 EVM 理解成确定性、gas-metered 的状态转换执行环境。

30 秒回答

EVM 是 Ethereum 的确定性虚拟机。每个节点对同一交易和同一前置状态执行同一 bytecode/opcode 序列，得到同一状态结果；它是 256-bit word 的 stack machine，并通过 gas 约束资源。

深入解析

把 EVM 看成 state transition function 更有用：输入包括当前 world state、交易环境和 calldata，执行过程中操作 stack、memory、storage、logs 与 call frames，最终提交或回滚状态变化。

Ethereum 官方文档明确指出 EVM stack 深度最多 1024 项，每项 256 bit；memory 仅在执行期间存在，而 storage 属于账户持久状态。当前 EVM 还包括 transaction-scoped transient storage。

必须说出的关键点

- deterministic execution
- 256-bit stack machine
- gas-metered
- state transition, not merely “a VM for Solidity”

高频追问

- Solidity 与 EVM 的关系？Solidity 编译成 EVM bytecode，但 EVM 不只支持 Solidity。
- EVM 为什么必须 deterministic？否则不同节点无法就 state root 达成一致。
- EVM 和 consensus layer 的边界是什么？执行判断交易结果，共识排序/最终化区块。

常见误区 / Gotchas

- 把 EVM 说成“一台运行在云上的虚拟机”。
- 把 EVM 与 Ethereum 共识协议混成一个概念。

记忆锚点 EVM = 每个验证节点都能重复执行并得到同一结果的 gas 计量状态机。

参考：R1 Ethereum EVM

Q12 Stack / Memory / Storage / Transient Storage 有什么区别?

难度 ★★★ | 高频度 ★★★★★ | 岗位 Solidity/安全

面试官意图 现代 EVM 高频题，尤其 2026 需要加入 transient storage。

30 秒回答

Stack 保存 256-bit 操作数；memory 是当前调用帧的临时字节数组；storage 是账户持久化状态；transient storage 在同一 transaction 的内部调用间共享，但交易结束自动清空。

深入解析

四者最重要的区别是生命周期、寻址方式、跨调用可见性和 gas 成本。Memory 在 call frame 内使用；storage 会写进全局状态 trie；transient storage 通过 TLOAD/TSTORE 访问，不进入持久 storage trie。

EIP-1153 的 transient storage 特别适合 transaction-scoped coordination，例如重入锁或临时授权，但它仍有 revert/DELEGATECALL 语义，不能简单当成“更便宜的 memory”。

必须说出的关键点

- Stack: operands, depth 1024
- Memory: frame-local temporary bytes
- Storage: persistent account state
- Transient: transaction-scoped key-value state

高频追问

- 为什么 transient storage 能用于 reentrancy guard？跨内部 call 可见且交易结束清空。
- delegatecall 下 transient storage 属于谁？与 storage context 类似，取决于执行上下文。
- memory 扩容为什么会越来越贵？存在 memory expansion cost。

常见误区 / Gotchas

- 把 transient storage 说成“跨交易缓存”。
- 认为 memory 可以被另一个 external call frame 直接共享。

记忆锚点 生命周期：stack/memory < call；transient = transaction；storage = 跨 transaction。

参考：R1 Ethereum EVM, R4 EIP-1153

Q13 Solidity 的 storage、memory、calldata 有什么区别?

难度 ★★ | 高频度 ★★★★★ | 岗位 Solidity

面试官意图 从 Solidity data location 推导复制成本、可变性和 ABI 输入。

30 秒回答

`storage` 引用持久状态；`memory` 是可修改的临时副本；`calldata` 是外部调用输入的只读区域。对于只读动态参数，直接使用 calldata 常避免一次 memory copy。

深入解析

引用类型必须明确 data location。函数接收大数组时，`calldata` 可以直接读取 ABI 编码输入；如果先复制到 memory，会产生额外成本。返回值通常需要编码进 returndata，因此“calldata 永远最省”也不是绝对规则。

还要注意 aliasing：`storage` 局部变量可能是指向状态的引用，修改它会直接修改持久状态；从 storage 赋值到 memory 则产生副本。

必须说出的关键点

- calldata immutable
- memory mutable temporary
- storage persistent reference
- 关注 copy vs reference 语义

高频追问

- 为什么 external function 常用 calldata?
- `bytes calldata` slice 有什么优势?
- storage reference 赋值时什么时候是 alias、什么时候是 copy?

常见误区 / Gotchas

- 只背“calldata 最便宜”而不说明何时会复制。
- 忽略 struct/array 引用语义导致意外状态修改。

记忆锚点 选择 data location 的核心问题是：这份数据要不要持久化、要不要修改、要不要复制。

Q14 CALL 与 DELEGATECALL 有什么区别?

难度 ★★★ | 高频度 ★★★★★ | 岗位 Solidity/安全/Proxy

面试官意图 Proxy、library 和 upgradeability 的核心题。

30 秒回答

`CALL` 在目标合约自己的 storage/context 中执行目标代码；`DELEGATECALL` 执行目标代码但使用调用者的 storage、address 和余额上下文，同时保留原始 msg.sender / msg.value 语义。

深入解析

Proxy 的本质就是“用户调用 proxy -> fallback -> delegatecall implementation”。代码来自 implementation，但写入的是 proxy storage，所以升级 implementation 可以保留地址、余额和状态。

风险也由此而来：implementation 的 storage layout、权限检查和初始化逻辑必须与 proxy 语义匹配。任意 delegatecall 到不可信地址近似于让别人以你的合约身份执行代码。

代码 / 状态机片段

```
// proxy fallback concept
(bool ok, bytes memory ret) = implementation.delegatecall(msg.data);
require(ok);
```

必须说出的关键点

- CALL: target code + target storage
- DELEGATECALL: target code + caller storage context
- proxy state lives in proxy
- msg.sender preservation 是 meta/proxy 语义关键

高频追问

- delegatecall 中 `address(this)` 是谁？调用者/proxy。
- 为什么 library 可以用 delegatecall？复用代码并作用于调用者上下文。
- delegatecall 到 selfdestruct 历史上为何危险？取决于当前 EVM 语义与硬分叉。

常见误区 / Gotchas

- 说“delegatecall 会继承余额”但不解释 storage context。
- 忽略 storage collision 与 initialization。

记忆锚点 CALL 是“去别人家执行”；DELEGATECALL 是“借别人的代码，在我家执行”。

Q15 ABI Function Selector 和 calldata 如何编码?

难度 ★★ | 高频度 ★★★★☆ | 岗位 Solidity/后端/安全

面试官意图 检查你能否从函数签名追到原始 calldata。

30 秒回答

外部函数调用的前 4 bytes 通常是 `keccak256("name(type1,type2,...)")` 的前 4 bytes，后面按 ABI 规则编码参数。动态类型的 head 区存 offset，tail 区存实际长度与数据。

深入解析

例如 `transfer(address,uint256)` 的 selector 固定为 4 bytes。因为 selector 只有 32 bit，理论上存在碰撞，所以 proxy / dispatcher 设计需要考虑 selector clash。

理解 ABI 能让你排查低级 call、decode revert data、构建 multicall、分析 calldata gas 和安全工具输出。

selector = first4(keccak256("f(type1,type2,...)"))

仅函数名不够，参数类型是签名的一部分

必须说出的关键点

- canonical type names 参与签名
- selector 只有 4 bytes
- static values inline, dynamic values use offset
- returndata / revert data 也遵循 ABI 约定

高频追问

- 为什么 `uint` 在签名里 canonicalize 为 `uint256`?
- custom error selector 怎么算? 与函数 selector 类似。
- abi.encode 与 abi.encodePacked 有何差异?

常见误区 / Gotchas

- 把 selector 当成完整 hash，忽略 4-byte 碰撞。
- 在签名字符串里错误加入参数名或空格。

记忆锚点 ABI 是“链上二进制协议”；selector 决定调用哪个入口，后续 bytes 决定参数。

Q16 Mapping 在 EVM Storage 中如何定位?

难度 ★★★ | 高频度 ★★★★★ | 岗位 Solidity/安全

面试官意图 从高级 Solidity 映射到底层 slot，是 proxy、取证和 storage collision 的基础。

30 秒回答

假设 mapping 本身声明在 slot p，键 k 对应的 value 通常位于 `keccak256(encode(k) || encode(p))` 得到的 storage slot；嵌套 mapping 会继续递归 hash。

深入解析

Mapping 不能像数组那样在 slot p 直接顺序存值，因为 key 空间巨大且稀疏。slot p 更像一个命名空间 seed。对动态数组、bytes/string 又有不同的 storage layout 规则。

掌握这个规则后，可以使用 `eth_getStorageAt`、Foundry cast 或调试器直接读取合约 raw storage；升级合约时也能理解为什么变量顺序变化会破坏解释。

slot(value[k]) = keccak256(encode(k) || encode(p))

p 为 mapping 声明所在 storage slot

必须说出的关键点

- mapping base slot p 不直接存元素
- element slot = keccak(key,p)
- nested mapping recursively hashes
- storage layout 是 ABI 之外的另一套关键规则

高频追问

- `mapping(address=>mapping(uint=>X))` 如何算 slot？先外层 hash，再内层。
- struct 作为 mapping value 如何定位成员？以元素起始 slot 加 member offset。
- 为什么 mapping 无法原生枚举 keys？链上不保存 key list。

常见误区 / Gotchas

- 对 key 使用错误 padding/encoding。
- 认为 `delete mapping` 可以自动遍历清空全部 key。

记忆锚点 Mapping 的 slot 是“哈希寻址命名空间”，不是顺序数组。

Q17 CREATE 与 CREATE2 有什么区别?

难度 ★★★ | 高频度 ★★★★☆ | 岗位 Solidity/钱包/协议

面试官意图 检查 deterministic deployment 与 counterfactual address 直觉。

30 秒回答

CREATE 的新地址主要由 deployer address 和 nonce 决定；CREATE2 则由 deployer、salt 与 initcode hash 决定，因此在真正部署之前就可以计算未来合约地址。

深入解析

CREATE2 使“先知道地址、后部署代码”成为可能，常见于 smart account factory、channel、clone factory 和 deterministic deployment。地址依赖 initcode hash，因此 constructor arguments 也会影响地址。

安全上必须理解 salt 并不是秘密，且预计算地址只承诺 initcode hash；部署流程还要考虑重复 salt、初始化、工厂权限和链间地址一致性。

addr = last20(keccak256(0xff || deployer || salt || keccak256(initcode)))

initcode 包含创建逻辑与 constructor 参数

必须说出的关键点

- CREATE: deployer + nonce
- CREATE2: 0xff + deployer + salt + keccak(initcode)
- counterfactual address
- initcode != runtime bytecode

高频追问

- 为什么 CREATE2 常用于 AA wallet？可以在部署前收到资产或签名引用。
- 同 salt 能否部署不同代码？地址还依赖 initcode hash。
- selfdestruct 后能否重新 CREATE2 同地址？必须结合当前 EVM 语义分析。

常见误区 / Gotchas

- 把 CREATE2 说成“用 salt 决定地址”而忽略 initcode。
- 把 runtime code hash 与 initcode hash 混淆。

记忆锚点 CREATE2 的核心不是“更安全”，而是把部署地址变成可预计算的函数。

Q18 EIP-1559 的费用机制如何工作?

难度 ★★★ | 高频度 ★★★★★ | 岗位 后端/钱包/协议

面试官意图 钱包估费、交易替换和区块空间经济学的核心。

30 秒回答

EIP-1559 把交易费拆为 protocol-determined base fee 和给 block producer 的 priority fee。用户设置 `maxFeePerGas` 与 `maxPriorityFeePerGas`，实际支付价格受当前 base fee 限制。

深入解析

Base fee 根据前序区块拥堵程度自动调整并被 burn，从而让用户不必完全用第一价格拍卖猜费。`maxFeePerGas` 是用户愿意支付的上限，不代表一定会支付到上限。

钱包还必须处理 replacement transaction：同 nonce 的新交易通常需要更高的有效费用参数才能替换 mempool 中旧交易。

effectiveTip = min(maxPriorityFee, maxFee - baseFee)

effectiveGasPrice = baseFee + effectiveTip

必须说出的关键点

- base fee is protocol-level
- priority fee incentivizes inclusion
- maxFee is cap, not exact price
- gas limit 与 gas price 完全不同

高频追问

- 有效 gas price 如何计算？约为 baseFee + min(maxPriority, maxFee-baseFee)。
- 为什么 base fee burn？协议设计的一部分，不直接支付 proposer。
- 交易长时间 pending 如何加速？发送同 nonce 更高费率替换交易。

常见误区 / Gotchas

- 把 maxFeePerGas 当实际单价。
- 忽略 blob fee 与 execution gas fee 是分离市场（EIP-4844）。

记忆锚点 1559 把“网络当前底价”和“我愿意额外付的小费”拆开。

Q19 EIP-1153 Transient Storage 解决了什么?

难度 ★★★★ | 高频度 ★★★☆☆ | 岗位 EVM/高级 Solidity

面试官意图 当前 EVM 新原语题，检查你是否跟上 Cancun 之后的执行模型。

30 秒回答

EIP-1153 引入 `TLOAD` / `TSTORE`。它像 storage 一样按 key/value 在同一 transaction 内跨调用共享，但交易结束时自动丢弃，因此适合不需要永久落盘的临时协作状态。

深入解析

传统 reentrancy lock 使用 SLOAD/SSTORE 修改持久 storage，即使最后恢复原值仍要承担状态访问成本。Transient storage 可以表达“只在当前 transaction 有意义”的 lock、temporary approval、callback context。

它不是 memory 的替代品：memory 属于具体 call frame，而 transient storage 与 storage context 相关，跨内部/外部调用的可见性和 revert 行为必须按 EIP 规则理解。

必须说出的关键点

- TLOAD/TSTORE
- cleared at end of transaction
- 100-gas-level warm-like access in spec
- 适合 transaction-scoped coordination

高频追问

- 为什么 Solidity 编译器可能提供 `transient` 变量支持？底层映射到 TLOAD/TSTORE。
- 重入锁如果使用 transient storage 有哪些 edge case？delegatecall/context 必须正确。
- 为何不能存长期 nonce？交易结束即消失。

常见误区 / Gotchas

- 把 transient storage 当“临时但跨 block”。
- 忽略低层库使用同 transient slot 的 collision。

记忆锚点 如果一个状态“只需要活到本笔交易结束”，优先思考 transient，而不是 persistent storage。

参考：R4 EIP-1153, R1 Ethereum EVM

Q20 一笔 Ethereum Transaction 从钱包到 Finality 经历什么?

难度 ★★★★ | 高频度 ★★★★★ | 岗位 全栈/后端/协议

面试官意图 系统级综合题，能迅速区分只会写合约和真正懂链路的人。

30 秒回答

钱包构造并签名交易，经 RPC 发送到节点，进入 mempool；交易被 builder/proposer 选入区块，execution client 执行 EVM 并生成 receipt/logs/state root；consensus layer 对区块完成确认与最终化，应用再按风险策略更新业务状态。

深入解析

完整链路至少包含：nonce/fee estimation -> signing -> JSON-RPC -> mempool -> block construction -> execution -> receipt/logs -> canonical chain -> finality。任何一层都可能失败，因此前端的“pending”不是单一状态。

高级回答应区分 execution failure 与 inclusion failure：交易可能签名正确但未被接收、在 mempool pending、被替换、被 reorg，或者被打包后 status=0。

必须说出的关键点

- transaction hash 只是 signed tx 标识，不保证 inclusion
- receipt 只有 inclusion 后才有
- logs 随 reorg 可能撤销
- 业务 finality policy 与链 finality 相关但不完全相同

高频追问

- 为什么 `eth_sendRawTransaction` 成功仍可能最终失败？只表示节点接受广播。
- 交易被 replacement 后原 hash 怎么处理？跟踪 nonce 与 replacement。
- L2 交易还要多哪些阶段？sequencer inclusion、batch posting、L1 settlement。

常见误区 / Gotchas

- 把“有 tx hash”当作支付成功。
- 只轮询 receipt，不处理 reorg/replacement。

记忆锚点 交易生命周期不是 Submitted -> Success 两个状态，而是一条带重试、替换、reorg 与 finality 的状态机。

CHAPTER 03

Solidity 工程：语言语义、ABI 与 Gas

本章目标 这一章不考“会不会写语法”，而是考语言语义在 EVM 上的成本、安全后果与接口边界。

本章题目

Q21 public / external / internal / private 有什么区别?
Q22 view 与 pure 有什么区别?
Q23 receive() 与 fallback() 有什么区别?
Q24 Solidity 0.8 之后为什么还会使用 unchecked?
Q25 immutable 与 constant 有什么区别?
Q26 Event 为什么通常比 storage 便宜？什么时候不能用 Event 替代 Storage?
Q27 Custom Error 为什么比 revert string 更适合生产合约?
Q28 ERC-20 Allowance Race Condition 是什么?
Q29 ERC-721 与 ERC-1155 的核心区别是什么?
Q30 如何系统做 Solidity Gas Optimization?

Q21 public / external / internal / private 有什么区别?

难度 ★★ | 高频度 ★★★★★ | 岗位 Solidity

面试官意图 基础题，但经常通过“private 是否保密”追问安全认知。

30 秒回答

`external` 主要供外部消息调用，`public` 同时生成外部入口且可被内部直接调用；`internal` 仅当前合约和继承链可见；`private` 仅当前合约代码可直接访问。可见性限制代码访问，不等于隐藏链上数据。

深入解析

状态变量标记 `private` 后，其他 Solidity 合约不能通过自动 getter 读取，但 storage 仍然存在公共链上，节点和观察者可以解析 raw storage。

函数可见性还会影响 ABI 与调用路径：内部调用 public function 可直接跳转，不必走 external message call；而 `this.f()` 会真正走外部调用并改变 msg.sender 语义。

必须说出的关键点

- private != confidential
- external/public 影响 ABI entrypoint
- internal 支持继承
- `this.f()` 与 `f()` 语义不同

高频追问

- 敏感随机数能否放 private state？不能。
- 为什么 external 参数常配 calldata？外部 ABI 输入可直接读取。
- private 函数能否被反编译发现？字节码公开，不能当秘密。

常见误区 / Gotchas

- 把 private 理解为加密。
- 只背可见性表，不说明外部调用边界。

记忆锚点 Solidity visibility 控制“谁能从代码层调用”，不控制“谁能看见链上数据”。

参考：R5 Solidity Security

Q22 view 与 pure 有什么区别?

难度 ★ | 高频度 ★★★★☆ | 岗位 Solidity

面试官意图 考察状态读取、编译器约束和 RPC `eth_call` 的区别。

30 秒回答

`view` 承诺不修改链上状态，但可以读取；`pure` 既不修改也不读取合约状态。它们是函数语义约束，不代表“如果在链上 transaction 中调用就不消耗执行资源”。

深入解析

前端通过 `eth_call` 执行 view/pure 时，由 RPC 节点本地模拟，不会把交易写入链，所以用户不支付链上手续费。但如果另一个合约在 transaction 内调用 view function，EVM 仍需执行其 opcodes 并消耗本笔交易的 gas。

`view` 对 external call 还可能通过 STATICCALL 施加只读限制，目标如果尝试修改状态会失败。

必须说出的关键点

- view can read state
- pure no state read/write
- eth_call no onchain fee != zero computation
- STATICCALL enforces no state writes

高频追问

- `block.timestamp` 在 pure 里能用吗？属于环境状态读取，不是 pure。
- view 函数里 emit event 可以吗？event 是状态外可观察副作用，编译器不允许。
- 为什么复杂 view 也可能 RPC 超时？节点仍实际执行计算。

常见误区 / Gotchas

- 说“view 不花 gas”而不区分 RPC 与链上调用。
- 把 pure 当“数学上无副作用”的完整保证，忽略 assembly/外部环境约束。

记忆锚点 view/pure 是代码语义；“免费”只是 eth_call 的业务支付方式。

Q23 receive() 与 fallback() 有什么区别?

难度 ★★ | 高频度 ★★★★★ | 岗位 Solidity/Proxy

面试官意图 基础入口题，常一路追问到 proxy fallback/delegatecall。

30 秒回答

`receive()` 主要处理空 calldata 的原生币接收；`fallback()` 在没有匹配函数 selector 时执行，并可在特定声明下接收 ETH。Proxy 通常利用 fallback 捕获任意 selector 后 delegatecall 到 implementation。

深入解析

函数分派首先尝试匹配 selector。如果 calldata 为空并存在 receive，通常进入 receive；否则无匹配时进入 fallback。一个没有 receive/fallback 的合约通过普通 message call 接收 ETH 会 revert。

安全上 receive/fallback 也是外部可控入口，应尽量简单。任何向未知地址转 ETH 的行为都可能触发目标合约代码，带来 callback/reentrancy。

必须说出的关键点

- empty calldata -> receive 优先
- unmatched selector -> fallback
- proxy fallback 是通用 dispatcher
- 收 ETH 也可能执行代码

高频追问

- `fallback(bytes calldata)` 可以返回 bytes 吗？现代 Solidity 支持特定形式。
- 为什么 transfer/send 的 2300 gas stipend 不能作为长期安全假设？gas schedule 会变化。
- Proxy admin 调 fallback 有何透明代理特殊规则?

常见误区 / Gotchas

- 把 fallback 说成“只在收 ETH 时触发”。
- 在 fallback 放复杂状态逻辑导致 DoS/gas 风险。

记忆锚点 receive 是“纯转账入口”，fallback 是“兜底消息入口”；Proxy 把兜底入口变成路由器。

参考：R5 Solidity Security

Q24 Solidity 0.8 之后为什么还会使用 unchecked?

难度 ★★ | 高频度 ★★★★☆ | 岗位 Solidity/Gas

面试官意图 考察算术安全和可证明的 gas 优化边界。

30 秒回答

Solidity 0.8 默认对整数 overflow/underflow 做 checked arithmetic，溢出会 revert。`unchecked {}` 显式关闭检查，可在已经由循环边界或业务 invariant 保证不会溢出的局部代码里节省少量 gas。

深入解析

常见例子是受数组长度约束的 `++i`：如果 `i < len` 且 len 远小于 uint256 最大值，开发者可以证明自增不会溢出。关键不是“unchecked 更快”，而是安全证明必须先于优化。

即使 checked arithmetic，也不能自动消灭所有数学漏洞：精度截断、先乘后除、signed/unsigned 转换、经济公式错误仍然可能导致严重问题。

必须说出的关键点

- default checked arithmetic
- unchecked is opt-out
- 证明安全边界再优化
- math correctness > overflow check

高频追问

- 为什么 unchecked 只在局部 block 使用更好？缩小审计范围。
- `type(uint256).max + 1` 在 unchecked 是什么？wrap to 0。
- 固定点数学的 rounding direction 为什么是安全问题？可能偏向攻击者。

常见误区 / Gotchas

- 看到 for-loop 就机械 unchecked。
- 认为 0.8 后再也不需要审计 arithmetic。

记忆锚点 unchecked 是“我能证明这里不会溢出”的声明，不是“我想省 gas”的按钮。

参考：R5 Solidity Security

Q25 immutable 与 constant 有什么区别?

难度 ★★ | 高频度 ★★★★☆ | 岗位 Solidity/Gas

面试官意图 考察部署期配置与 runtime storage 成本。

30 秒回答

`constant` 的值在编译期已知并直接嵌入字节码；`immutable` 可以在 constructor 阶段根据部署参数赋值一次，之后同样不可修改，通常通过 code 中的 immutable data 读取而不占普通 storage slot。

深入解析

如果某个地址每个部署实例不同，例如 token/oracle/factory，immutable 很合适；如果值是协议全局固定数学常数，constant 更自然。Upgradeability 场景需要格外注意 constructor 与 implementation/proxy 语义。

“不可修改”并不自动代表不可替换：如果合约本身是 proxy，implementation 代码升级仍可能改变读取和使用这些值的逻辑。

必须说出的关键点

- constant compile-time
- immutable constructor-time
- both cheaper than repeated SLOAD in many cases
- upgradeable contract constructor caveat

高频追问

- 为什么 upgradeable implementation 常用 initializer 而不是 constructor？proxy storage 不会执行 implementation constructor。
- immutable 能否每个 proxy 不同？通常 implementation immutable 对共享 implementation 是相同的，需谨慎。
- constant array/struct 有何限制？看编译器支持。

常见误区 / Gotchas

- 把 immutable 当 storage 里的只读变量。
- 在 proxy 系统里忽略 implementation 部署期状态。

记忆锚点 constant 在“编译时固定”，immutable 在“部署时固定”。

Q26 Event 为什么通常比 storage 便宜？什么时候不能用 Event 替代 Storage?

难度 ★★★ | 高频度 ★★★★★ | 岗位 Solidity/后端

面试官意图 测试链上状态与链下索引的边界。

30 秒回答

Event 写入 transaction logs，主要供链下观察与索引；storage 是合约后续执行可读取的持久状态。日志不进入合约可随机访问的 storage，因此通常更便宜，但不能替代合约未来需要参与判断的状态。

深入解析

典型模式是“storage 保存最小当前状态，event 保存历史轨迹”。例如订单是否已成交必须能在链上判断，不能只 emit `Filled`；而成交明细、UI 历史则适合通过 logs 重建。

Indexer 依赖 event 时要处理 reorg、ABI 版本、重复消费和 backfill。Event 不是数据库事务日志的完美替代，因为它跟随 canonical chain。

必须说出的关键点

- logs for offchain observation
- storage for future onchain decisions
- indexed topics improve filtering
- event history must be reorg-aware

高频追问

- 为什么合约不能读取过去 event？EVM 没有通用历史 log 查询 opcode。
- indexed 参数越多越好吗？有额外成本且 topic 数有限。
- 只靠 event 重建余额安全吗？作为 UI 可以，作为链上资金判断不行。

常见误区 / Gotchas

- 用 event 代替必须防 double-spend 的状态。
- 后端看到 event 就永久落账，忽略 reorg。

记忆锚点 Event 记录“发生过什么”，Storage 记录“合约现在需要知道什么”。

Q27 Custom Error 为什么比 revert string 更适合生产合约?

难度 ★★ | 高频度 ★★★★☆ | 岗位 Solidity/Gas

面试官意图 考察 error ABI、部署字节码和可观测性。

30 秒回答

Custom error 通过 4-byte selector + ABI 参数编码错误，通常比长 revert string 更紧凑，减少部署字节码和 revert data 成本，同时可以携带结构化上下文。

深入解析

例如 `error Unauthorized(address caller);` 比 `require(ok, "caller is not authorized")` 更适合机器解码。前端和测试可以根据 selector 精确断言错误类型。

但不要为了省 gas 删除有价值的错误语义。好的 error 名称和参数实际上提高审计、monitoring 和 debug 能力。

代码 / 状态机片段

```
error Unauthorized(address caller);

if (msg.sender != owner) {
    revert Unauthorized(msg.sender);
}
```

必须说出的关键点

- structured revert ABI
- 4-byte selector
- smaller bytecode/revert data
- better test/debug semantics

高频追问

- custom error selector 如何算？与函数一样 hash 签名取前 4 bytes。
- low-level call 如何拿到 error data？返回的 returndata。
- panic code 与 custom error 有什么不同？panic 由 Solidity 内部错误产生。

常见误区 / Gotchas

- 所有错误都只用 `error E();` 导致可观测性下降。
- 认为 revert 不消耗任何 gas；之前执行的 gas 仍然消耗。

记忆锚点 Custom Error 把“给人看的字符串”升级成“给程序和人都能解码的结构化错误”。

Q28 ERC-20 Allowance Race Condition 是什么?

难度 ★★★ | 高频度 ★★★★★ | 岗位 Solidity/DeFi/安全

面试官意图 经典 token 授权题，考 transaction ordering 与授权状态机。

30 秒回答

当 owner 想把某 spender 的 allowance 从旧值改为新值时，spender 可能观察 pending approve 并先使用旧额度，再让新的 approve 生效，从而在某些顺序下消费超过 owner 原本直觉上的目标。

深入解析

问题来自 `approve(spender, newAmount)` 是覆盖式赋值，而交易顺序由链决定。传统 UI 常建议先把 allowance 设为 0 再设新值，或者使用 increase/decrease allowance、permit、一次性授权等更明确的授权模型。

生产设计还要考虑无限授权的 blast radius：即使 ERC-20 本身没有 bug，被授权的 router/contract 一旦被攻破，用户资金也可能受影响。

必须说出的关键点

- overwrite allowance + mempool ordering
- zero-first is mitigation, not universal cure
- permit shifts approval to signed message
- 授权范围最小化

高频追问

- 为什么 `transferFrom` 需要 allowance？delegated spending。
- permit 是否消除所有风险？仍要 nonce/domain/deadline 和 spender 安全。
- temporary approval 有何价值？缩小授权生命周期。

常见误区 / Gotchas

- 只说“ERC20 有 front-running”但解释不出状态变化。
- 默认无限 approve 是最佳 UX 而不讨论风险。

记忆锚点 Allowance 不是“给一次许可”，而是一个可被并发交易修改和消费的链上额度状态。

Q29 ERC-721 与 ERC-1155 的核心区别是什么?

难度 ★★ | 高频度 ★★★★☆ | 岗位 Solidity/NFT

面试官意图 考 token 标准抽象和 batch efficiency。

30 秒回答

ERC-721 主要表示每个 tokenId 独立唯一的 NFT 所有权；ERC-1155 在一个合约中统一表示多种 token type，每个 id 可有数量，因此既可表达 fungible 也可表达 non-fungible，并原生支持 batch 操作。

深入解析

ERC-1155 的 `balanceOf(account,id)` 模型适合游戏道具、半同质资产和多资产仓库；ERC-721 的 `ownerOf(tokenId)` 更直观地表达唯一资产。

面试可继续讨论 safe transfer callback：向合约发送 NFT/token 时，接收方接口回调是 composability 机制，也同时是潜在重入入口。

必须说出的关键点

- 721: one owner per tokenId
- 1155: balances per (account,id)
- 1155 batch transfer
- receiver callbacks are security boundary

高频追问

- ERC721A 解决什么问题？主要优化批量 mint/storage，不是新标准语义。
- NFT metadata 在链上还是链下？tokenURI 可指向外部内容，信任模型不同。
- safeTransferFrom 为什么比 transferFrom 更安全？检查合约接收接口。

常见误区 / Gotchas

- 把 1155 说成“只能 fungible”。
- 忽略 callback 对重入分析的影响。

记忆锚点 721 的核心是 ownerOf；1155 的核心是 balance(account,id)。

Q30 如何系统做 Solidity Gas Optimization?

难度 ★★★ | 高频度 ★★★★★ | 岗位 Solidity/高级

面试官意图 高级候选人要能按成本大头排序，而不是背微优化清单。

30 秒回答

先优化算法和状态写入次数，再考虑 data location、storage caching、packing、custom errors、immutable、batch 和安全可证的 unchecked。Gas 优化的优先级应是“减少持久状态与外部交互 > 减少复制/重复读取 > 微观 opcode 技巧”。

深入解析

最大的浪费往往不是 `i++` 和 `++i`，而是 O(N) 状态循环、重复 SSTORE、逐条外部 call、无法批处理的数据模型。先用 profiler / gas snapshot 找热点，再优化。

Storage packing 只在访问模式匹配时有意义：把多个小变量压进同一 slot 可以减少 slot 数，但如果每次只改单个字段，仍要 read-modify-write；复杂 packing 还可能降低可读性。

必须说出的关键点

- measure first: gas snapshot
- reduce SSTORE / external calls
- calldata + cache storage
- packing/unchecked only after correctness

高频追问

- 为什么 deleting storage 有时有 refund？规则会随 hard fork 调整，不能只背历史常数。
- batching 如何同时降低 base tx cost 和 external call overhead?
- 优化 proxy 与 immutable 时有何冲突?

常见误区 / Gotchas

- 为了省几十 gas 使用难审计 assembly。
- 没有 benchmark 就宣称某写法“更省”。

记忆锚点 Gas 优化先看复杂度和状态模型，最后才看语法微优化。

CHAPTER 04

智能合约安全：从漏洞列表到 Invariant

本章目标 真正的安全面试不是背 SWC 编号，而是识别资产、信任边界、状态不变量、外部调用和经济假设。

本章题目

Q31 什么是 Reentrancy?
Q32 Checks-Effects-Interactions 为什么有效?
Q33 ReentrancyGuard 是否解决所有 Reentrancy?
Q34 为什么不能使用 tx.origin 做 Authorization?
Q35 Oracle Manipulation 的根因是什么?
Q36 Flash Loan 为什么不是漏洞本身?
Q37 Access Control 应如何设计?
Q38 Transparent Proxy 与 UUPS Proxy 有什么区别?
Q39 Upgradeable Contract 为什么不能随便改变 State Variable 顺序?
Q40 给你一份 200 行 Solidity 合约，如何系统做 Security Review?

Q31 什么是 Reentrancy?

难度 ★★ | 高频度 ★★★★★ | 岗位 Solidity/安全/DeFi

面试官意图 第一安全题，要求从控制流与状态提交顺序解释。

30 秒回答

Reentrancy 是合约在完成关键状态更新之前把控制权交给外部地址，外部代码又回调原合约或相关合约，使同一逻辑在旧状态上再次执行。

深入解析

任何 external call 都可能交出控制权，不只 ETH transfer。经典 withdraw bug 是先发送资金、后把余额清零；攻击者回调时仍看到旧余额，于是重复提取。

现代分析要超越“同一函数重入”，还包括 cross-function、cross-contract、callback token、read-only reentrancy 等。关键判断不是函数名，而是“外部交互期间哪些 invariant 暂时被破坏”。

必须说出的关键点

- external call = control transfer
- state effects before interaction
- cross-function/cross-contract reentrancy
- callbacks from ERC777/NFT/DeFi hooks

高频追问

- 为什么 ERC20 transfer 也不能绝对假设无 callback？可能是非标准 token/proxy。
- read-only reentrancy 是什么？在临时不一致状态读取错误价格/份额。
- 为什么 nonReentrant 不是完整方案？见 Q33。

常见误区 / Gotchas

- 只把重入理解成 fallback 重复 withdraw。
- 看到 transfer 就认为安全，因为 2300 gas 历史假设。

记忆锚点 重入的本质：你还没把账本改完，就让不可信代码拿到了执行权。

参考：R5 Solidity Security

Q32 Checks-Effects-Interactions 为什么有效?

难度 ★★ | 高频度 ★★★★★ | 岗位 Solidity/安全

面试官意图 确认能把安全模式对应到状态机原理。

30 秒回答

CEI 要求先检查前置条件，再写入本合约状态，最后进行外部交互。这样 callback 即使发生，也会观察到已经提交的新状态，而不是旧状态。

深入解析

CEI 相当于尽量把本合约 invariant 在“交出控制权”之前恢复到安全状态。例如提现时先读取 amount 并将余额置零，再 call 用户。

但 CEI 不是所有重入的万能证明。多个合约共享 invariant、外部调用前必须获取返回值、或者 read-only reentrancy 时，仍需锁、pull pattern、架构拆分和 invariant tests。

代码 / 状态机片段

```
uint256 amount = balance[msg.sender];
require(amount != 0);     // Check
balance[msg.sender] = 0;  // Effect
(bool ok,) = msg.sender.call{value: amount}(""); // Interaction
require(ok);
```

必须说出的关键点

- Checks -> Effects -> Interactions
- restore local invariant before call
- pull payment often safer
- CEI + guard + design-level invariant

高频追问

- 如果 external call 失败，前面的 effects 会怎样？同一调用 revert 时会回滚。
- 为什么 pull over push 能降低 DoS？每个用户独立提现。
- 跨合约 invariant 如何保护？需要更高层锁或原子状态设计。

常见误区 / Gotchas

- 把 CEI 当成“有了就不用 ReentrancyGuard”。
- 先 emit event 后 effect 是否算 interaction？event 不交控制权，但语义上最好反映最终状态。

记忆锚点 在把控制权交出去之前，让自己的账本先达到一个自洽状态。

参考：R5 Solidity Security

Q33 ReentrancyGuard 是否解决所有 Reentrancy?

难度 ★★★ | 高频度 ★★★★★ | 岗位 安全/DeFi

面试官意图 测试候选人是否会把安全库当万能药。

30 秒回答

不会。单一 `nonReentrant` 锁可以阻止受保护入口在同一锁域重入，但可能遗漏未加锁的相关函数、跨合约共享状态、只读价格查询、回调次序和 delegatecall 上下文。

深入解析

典型 cross-function 场景：`withdraw()` 加锁，但 `transferBalance()` 没加锁且读取同一中间状态；攻击者从 callback 进入另一个函数仍可能破坏 invariant。

此外，ReentrancyGuard 只限制控制流，不证明经济账本正确。如果状态更新顺序本身错误、oracle 可操纵或 share accounting 错误，加再多锁也无济于事。

必须说出的关键点

- guard scope must cover shared invariant
- read-only reentrancy exists
- cross-contract locks may be needed
- CEI remains valuable

高频追问

- 为什么两个 `nonReentrant` external functions 不能直接互调？同一锁会阻塞，可拆 internal function。
- transient reentrancy guard 有什么优势？降低持久 storage 成本。
- 如何用 invariant fuzz 发现重入？构造 adversarial receiver。

常见误区 / Gotchas

- “加 OpenZeppelin ReentrancyGuard 就安全了”。
- 只审函数级锁，不审资产级 invariant。

记忆锚点 锁只能限制“谁什么时候进来”，不能替你证明“账本为什么正确”。

Q34 为什么不能使用 tx.origin 做 Authorization?

难度 ★★ | 高频度 ★★★★★ | 岗位 Solidity/安全

面试官意图 经典 phishing-style call-chain 漏洞。

30 秒回答

`tx.origin` 是整笔交易最初的 EOA，而 `msg.sender` 是当前这一跳调用者。如果 owner 被诱导调用恶意合约，恶意合约再调用受害合约，tx.origin 仍然是 owner，错误的授权检查会被绕过。

深入解析

授权应该验证当前直接调用者或使用明确签名/role 机制。Solidity 官方安全文档直接建议“Never use tx.origin for authorization”。

EIP-7702 之后更应该减少对 `tx.origin == msg.sender` 一类 EOA 假设的依赖，因为 EOA 可以设置委托代码，账户行为比历史模型更复杂。

必须说出的关键点

- tx.origin = transaction origin EOA
- msg.sender = immediate caller
- authorization follows local call boundary
- 7702 makes EOA assumptions even less reliable

高频追问

- tx.origin 有完全不能用的场景吗？可用于非授权观察，但通常应谨慎。
- 为什么 `require(tx.origin == msg.sender)` 作为 anti-bot 很脆弱？破坏 composability 且账户模型变化。
- meta-transaction 下 msg.sender 怎么处理？通过 trusted forwarder/context。

常见误区 / Gotchas

- 认为 tx.origin 更“真实”，所以更安全。
- 用 tx.origin 阻止合约调用，导致 AA/DeFi 可组合性失败。

记忆锚点 授权看“谁直接调用我”，不要看“这条交易最早是谁发起”。

参考：R5 Solidity Security, R3 EIP-7702

Q35 Oracle Manipulation 的根因是什么?

难度 ★★★ | 高频度 ★★★★★ | 岗位 DeFi/安全

面试官意图 经济安全核心题：价格不是一个函数返回值，而是带流动性、延迟与信任假设的数据。

30 秒回答

Oracle manipulation 通常发生在协议把可被短期资本影响的价格源当作可信公允价，攻击者通过交易改变 spot price，再利用 lending、mint、liquidation 等逻辑按错误价格结算。

深入解析

风险取决于操纵成本与可提取价值。一个低流动性 AMM 的瞬时 spot price 可能只需要很少资本就能推高；flash loan 进一步降低“必须长期持有资本”的门槛。

防御组合包括高质量外部 oracle、TWAP、最小流动性、staleness check、deviation bounds、多源聚合、circuit breaker 和对新资产设置保守参数。

必须说出的关键点

- price source trust model
- manipulation cost vs extractable value
- freshness + deviation + liquidity checks
- TWAP is mitigation, not magic

高频追问

- TWAP 为什么更难操纵？需要跨时间维持价格偏离。
- Chainlink/外部 oracle 也可能有什么风险？stale、feed misconfiguration、L2 sequencer downtime。
- stablecoin depeg 时按 1 美元定价有什么风险?

常见误区 / Gotchas

- “用了 oracle 就安全”。
- 只看价格精度，不看更新频率与 failure mode。

记忆锚点 Oracle 安全的核心公式：攻击成本必须高于利用错误价格能提取的价值。

Q36 Flash Loan 为什么不是漏洞本身？

难度 ★★★ | 高频度 ★★★★★ | 岗位 DeFi/安全

面试官意图 区分攻击工具与被攻击的脆弱假设。

30 秒回答

Flash loan 只是允许在一笔原子交易中无抵押借入大量流动性并在结束前归还。它让攻击者暂时拥有大资本，但真正漏洞来自可操纵 oracle、错误 accounting、治理快照或其他可被原子资本放大的机制。

深入解析

传统金融里“大资本门槛”可能是隐性防线；flash loan 让这个假设消失。所以协议设计必须假设攻击者在单笔交易内可以获得几乎任意大的可借流动性，只要有可用池。

同样，flash loan 也有正常用途：套利、再融资、清算、抵押品迁移。安全评估应关注“如果攻击者突然获得很大原子资本，我的 invariant 还成立吗？”

必须说出的关键点

- atomic capital
- tool != vulnerability
- remove capital-availability assumptions
- stress test economic invariants

高频追问

- 为什么 flash loan 必须同交易归还？否则整笔交易 revert。
- 治理为什么要用历史 snapshot？避免临时借票权。
- oracle manipulation 和 flash loan 的组合路径怎么分析？从资本 -> 价格 -> 借贷额度 -> 提现。

常见误区 / Gotchas

- 把 flash loan 全部描述成恶意功能。
- 修复方案只是“禁止 flash loan”，但无法识别借来的普通资本。

记忆锚点 不要防 flash loan；要让协议在“攻击者有无限临时资本”的假设下仍然安全。

Q37 Access Control 应如何设计?

难度 ★★★ | 高频度 ★★★★★ | 岗位 Solidity/安全/治理

面试官意图 判断能否把最小权限原则落实到角色、multisig、timelock 和 emergency powers。

30 秒回答

简单合约可用 single owner；复杂协议应把 mint、pause、upgrade、parameter change、treasury 等能力拆成独立 roles，并让高风险权限由 multisig/timelock/governance 管理。

深入解析

权限设计首先做 capability inventory：每个 privileged function 能对用户资产造成什么最大影响？然后用 least privilege、role separation 和 time delay 降低单点失误。

OpenZeppelin 也强调 access control 是智能合约安全核心。工程上还应监控角色变更事件、设置两步 ownership transfer、限制 emergency role 的能力和生命周期。

必须说出的关键点

- least privilege
- separate ADMIN/MINTER/PAUSER/UPGRADER
- multisig + timelock
- monitor role changes

高频追问

- `DEFAULT_ADMIN_ROLE` 为什么危险？它能管理其他角色，权限极高。
- pause 权限是否应该能转移资产？通常应分离。
- 为什么 timelock 提高安全？给用户/监控系统发现恶意变更的时间。

常见误区 / Gotchas

- 所有权限都给一个 owner EOA。
- 为了“去中心化”完全删除应急机制但没有 permissionless exit。

记忆锚点 先列“谁能做什么”，再谈“是不是去中心化”。权限边界本身就是协议的一部分。

参考：R6 OpenZeppelin Access Control

Q38 Transparent Proxy 与 UUPS Proxy 有什么区别?

难度 ★★★★ | 高频度 ★★★★★ | 岗位 Solidity/高级/安全

面试官意图 升级合约面试核心。

30 秒回答

Transparent Proxy 把 admin/upgrade dispatch 主要放在 proxy；UUPS 把 upgrade mechanism 放在 implementation，并要求 `_authorizeUpgrade` 正确限制升级权限。两者常基于 ERC-1967 implementation slot。

深入解析

Transparent pattern 为避免 selector clash，对 admin 与普通用户采用不同 fallback 行为；UUPS proxy 本身更轻，implementation 负责 upgrade logic，因此部署成本更低、灵活性更高。

OpenZeppelin 当前文档明确说明其推荐趋势偏向 UUPS，同时警告 upgradeable proxy 需要深度理解 EVM。UUPS 的关键安全点是 implementation 必须保留正确的授权与 proxiable 检查，否则可能丢失升级能力或被未授权升级。

必须说出的关键点

- state lives in proxy
- Transparent: upgrade/admin logic in proxy path
- UUPS: upgrade logic in implementation
- ERC-1967 slots avoid collisions

高频追问

- 为什么实现合约不能忘记 initializer lock？攻击者可能初始化 implementation 本身。
- UUPS `_authorizeUpgrade` 应如何保护？role/multisig/governance。
- Beacon proxy 适合什么场景？多个 proxies 共享一个 beacon implementation pointer。

常见误区 / Gotchas

- 认为 proxy “把数据也代理到 implementation”。
- 混用 UUPS implementation 与 Transparent proxy 而不了解权限风险。

记忆锚点 Transparent 是“代理管升级”，UUPS 是“实现管升级”；两者状态都在 proxy。

参考：R7 OpenZeppelin Proxy

Q39 Upgradeable Contract 为什么不能随便改变 State Variable 顺序?

难度 ★★★★ | 高频度 ★★★★★ | 岗位 Solidity/Proxy/安全

面试官意图 检验 storage layout 是否真正理解。

30 秒回答

Proxy 通过 delegatecall 用新 implementation 解释旧 proxy storage。如果 V2 改变已有变量的 slot/offset，新代码会把旧字节解释成另一个字段，造成 owner、balance、mapping seed 等全部错位。

深入解析

安全升级通常要求保持既有变量布局兼容，只在末尾追加新字段，或采用经过验证的 namespaced storage 模式。继承层级变化也可能移动变量，因此不能只看单个文件。

OpenZeppelin Upgrades Plugins 会做 storage compatibility 检查，但工具不是替代理解：自定义 assembly slots、delegatecall libraries、struct layout 变化仍需要人工审计。

必须说出的关键点

- proxy storage persists across versions
- existing slot meanings must not change
- inheritance affects layout
- automated validation + manual review

高频追问

- 删除一个变量再加新变量为什么危险？旧 slot 数据仍然存在且新字段可能复用。
- mapping/array 变量顺序变化为何更严重？base slot 是寻址 seed。
- 如何用 ERC-7201 namespaced storage 降低模块碰撞?

常见误区 / Gotchas

- “变量名没变就安全”；storage 只看 slot/offset，不看名字。
- 从 4.x implementation 直接升级到 5.x library 版本而不验证 layout。

记忆锚点 升级合约不是替换数据库 schema，而是让新代码继续解释同一块旧 storage。

参考：R8 OpenZeppelin Upgrades FAQ

Q40 给你一份 200 行 Solidity 合约，如何系统做 Security Review?

难度 ★★★★★ | 高频度 ★★★★★ | 岗位 安全/Senior/Protocol

面试官意图 综合安全能力题，重点是方法论而非漏洞清单。

30 秒回答

先建立资产、角色、外部入口、状态机和不变量，再按每个外部调用路径分析权限、资金流、oracle、callback、算术、upgrade、DoS 和 token compatibility，最后用测试、静态工具和 fuzz 验证假设。

深入解析

第一步不是跑 Slither，而是理解 specification：哪些资产必须永远守恒？哪些操作只能谁执行？哪个状态组合绝不能出现？如果不先写 invariant，工具输出只是噪声。

第二步构造 threat model：攻击者可以控制 calldata、交易顺序、callback、临时资本、部分外部依赖和时间。第三步逐个 entrypoint 做状态转换表，特别标记 external calls 前后的中间状态。

最后用工具增强人工审计：unit tests 覆盖正常路径，fuzz 覆盖边界，invariant tests 连续调用随机 action，static analyzer 找模式，fork tests 验证真实 token/oracle 兼容。

必须说出的关键点

- assets + trust boundaries
- invariants before vulnerabilities
- entrypoint/state-transition map
- external calls/oracles/upgrades are hotspots
- tooling validates hypotheses, not replaces reasoning

高频追问

- 你会优先看哪 5 个函数？所有能移动资产/改变权限/升级/改价格参数的入口。
- 如何审计一个陌生 DeFi 协议？先把 accounting equations 写出来。
- 测试通过是否代表安全？不代表，只说明覆盖到的性质成立。

常见误区 / Gotchas

- 先跑自动扫描再理解协议。
- 逐行找“危险函数名”，但没有系统级 invariant。

记忆锚点 安全审计的顺序：资产 -> 权限 -> 不变量 -> 状态机 -> 外部依赖 -> 对抗测试。

CHAPTER 05

DeFi：AMM、Lending 与经济不变量

本章目标 DeFi 面试看的是“公式如何成为资金状态机”。你需要从定价曲线、份额 accounting、清算与 oracle 风险推导系统行为。

本章题目

Q41 Uniswap V2 为什么使用 x*y=k?
Q42 Slippage 与 Price Impact 有什么区别?
Q43 什么是 Impermanent Loss?
Q44 Uniswap V3 Concentrated Liquidity 解决了什么问题?
Q45 Uniswap V3 Tick 与 sqrtPriceX96 是什么?
Q46 Lending Protocol 的 Health Factor 是什么?
Q47 为什么 DeFi Lending 通常需要 Over-collateralization?
Q48 Liquidation 的经济机制是什么?
Q49 Stablecoin 有哪些主要设计与 Failure Modes?
Q50 设计 DeFi Protocol 时最重要的 Invariant 是什么?

Q41 Uniswap V2 为什么使用 x*y=k?

难度 ★★ | 高频度 ★★★★★ | 岗位 DeFi/Solidity

面试官意图 确认 AMM 基础：价格来自曲线和储备变化，而不是合约里存一个 price 变量。

30 秒回答

Constant Product AMM 维持 `x*y=k`（考虑手续费时 k 通常非减）。交易者向池子加入一种资产并取出另一种资产，沿双曲线移动；边际价格近似由储备比决定，因此交易规模越大，price impact 越明显。

深入解析

在无手续费的连续模型中，从 reserve `(x,y)` 到 `(x+dx,y-dy)` 要满足乘积不变。求解 `dy` 可以看到输出是非线性的，这正是自动做市的定价机制。

协议不需要 order book 对手方；LP 预先提供两边库存。代价是 LP 承担库存再平衡和相对价格变化风险。

$$x * y = k$$

边际价格近似由 reserve ratio 决定

必须说出的关键点

- curve determines price
- large trade => nonlinear price impact
- fee changes effective input and increases LP value
- reserve ratio is not an external oracle

高频追问

- 为什么不能把 AMM spot price 直接用作 lending oracle？可被短期交易操纵。
- k 是否真的永远严格不变？手续费与实现细节下通常会增加。
- 如何推导 amountOut？将扣费后的 dx 代入不变量。

常见误区 / Gotchas

- 把 `x*y=k` 说成“保证价格不变”。
- 忽略 token fee-on-transfer 会破坏朴素 reserve accounting。

记忆锚点 AMM 不存“市场价”；它存库存，并用不变量把库存变化转成价格。

Q42 Slippage 与 Price Impact 有什么区别?

难度 ★★ | 高频度 ★★★★★ | 岗位 DeFi/前端

面试官意图 这是 DEX UI、交易保护与 MEV 的连接题。

30 秒回答

Price impact 是你自己的交易沿 AMM 曲线改变价格；slippage 更广义，表示提交时预期价格与最终执行价格之间的偏差，既可能来自自身 price impact，也可能来自其他交易、市场移动和排序。

深入解析

前端通常把 quote 输出与 `amountOutMin` / slippage tolerance 绑定。Tolerance 太紧会因正常价格移动频繁 revert；太宽则给不利排序或极端价格变化留下更大空间。

高级回答应区分 deterministic quote impact 和 inclusion-time uncertainty。交易在 mempool 等待期间，池状态可能已经变化。

必须说出的关键点

- price impact = your size vs liquidity
- slippage = expected vs executed
- minOut/deadline are execution guards
- wide slippage increases adverse execution risk

高频追问

- 为什么大池同金额 price impact 更小？深度更大。
- 限价单如何避免无限 slippage？只有满足价格才成交。
- MEV protection 能否替代 minOut？不能，仍应有链上约束。

常见误区 / Gotchas

- 把 slippage tolerance 理解成“手续费”。
- 设置 100% slippage 只是为了“保证成功”。

记忆锚点 Price impact 是曲线数学；slippage 是交易从报价到落链之间的现实误差。

Q43 什么是 Impermanent Loss?

难度 ★★★ | 高频度 ★★★★☆ | 岗位 DeFi

面试官意图 考 LP 经济直觉，而不是背百分比表。

30 秒回答

当池外相对价格变化时，套利者会交易 AMM 直到池内价格重新接近外部市场，LP 的两种资产库存随之再平衡。与“什么都不做、原地持有”相比，LP 组合价值可能更低，这部分差异常称 impermanent loss。

深入解析

IL 不是简单“池子亏钱”，而是一个 benchmark-relative 概念。LP 同时赚取手续费，所以最终收益取决于价格路径、波动、成交量和 fee income。

从期权视角，恒定乘积 LP 具有持续卖出上涨资产、买入下跌资产的再平衡特征，因此对波动存在特定风险暴露。

必须说出的关键点

- relative-to-HODL benchmark
- arbitrage rebalances inventory
- fees can offset IL
- path/volatility/liquidity range matter

高频追问

- 价格回到原点，IL 一定完全消失吗？理想模型接近，但手续费和路径改变最终结果。
- 集中流动性为什么可能放大 IL？更高资本效率也意味着更集中价格风险。
- StableSwap curve 如何改变这一权衡？

常见误区 / Gotchas

- 把 IL 当作“只要不撤池就没有损失”。
- 忽略 LP fees。

记忆锚点 LP 不是被动持有两种币，而是在曲线上持续自动再平衡库存。

Q44 Uniswap V3 Concentrated Liquidity 解决了什么问题?

难度 ★★★ | 高频度 ★★★★★ | 岗位 DeFi/协议

面试官意图 从 V2 低资本效率推导 V3 position model。

30 秒回答

V2 的流动性分布覆盖 `(0, ∞)` 全价格区间，大量资产在远离当前价格处几乎不会成交。V3 允许 LP 把资金集中在指定价格区间，以同样资本提供更深的局部流动性。

深入解析

集中流动性把 LP position 从“同质化池份额”变成带价格范围的个体 position。当前价格在区间内时 position 同时持有两资产并赚费；超出区间后逐渐变成单边资产并停止提供活跃流动性。

这提高资本效率，但带来主动管理、range selection、tick crossing 和更复杂的 fee accounting。Uniswap v3 白皮书把 concentrated liquidity 作为定义性创新。

必须说出的关键点

- range-bounded liquidity
- capital efficiency
- position becomes non-fungible configuration
- out-of-range liquidity is inactive

高频追问

- 为什么 V3 LP token 不再天然 fungible？range/fee tier 不同。
- 窄区间为什么收益和风险都更高？单位资本深度高但更易出区间。
- active liquidity 如何随 tick crossing 变化?

常见误区 / Gotchas

- 只说“V3 更省 gas”。
- 认为集中流动性总是优于 V2；它提高管理复杂度和风险集中度。

记忆锚点 V2 把钱铺满全价格轴；V3 允许 LP 把钱放在真正需要成交的局部。

参考：R9 Uniswap v3 Whitepaper

Q45 Uniswap V3 Tick 与 sqrtPriceX96 是什么?

难度 ★★★★ | 高频度 ★★★★☆ | 岗位 DeFi/高级

面试官意图 检查是否真正理解 V3 核心状态表达。

30 秒回答

V3 用离散 tick 表示指数价格网格，常写作 `P(i)=1.0001^i`；核心合约用固定点格式保存 `sqrt(price)`，即 `sqrtPriceX96`，便于在流动性公式中进行高精度整数运算。

深入解析

Liquidity 与 token0/token1 数量的公式天然包含 sqrt(P)，直接存 sqrt price 可减少反复开方并让整数数学更稳定。Q96 表示用 96 fractional bits 的定点数。

Tick 不是“价格精度小数位”，而是离散对数价格索引。fee tier 决定 tick spacing，只有特定 tick 可作为 position 边界。

P(i) = 1.0001^i ; sqrtPriceX96 ≈ sqrt(P) * 2^96

实际价格还需处理 token decimals

必须说出的关键点

- tick = logarithmic price index
- sqrtPriceX96 = sqrt(P) * 2^96
- tick spacing depends on pool config
- integer fixed-point math

高频追问

- 为什么 token0/token1 顺序会影响 price 定义?
- 如何从 sqrtPriceX96 还原 spot price? 平方后考虑 token decimals。
- tick 与 sqrtPrice 是否永远完全一致? 当前价格可能位于 tick 区间内。

常见误区 / Gotchas

- 直接把 sqrtPriceX96 除以 2^96 当 price，忘记平方。
- 忽略 token decimals。

记忆锚点 Tick 管“价格落在哪个对数格子”，sqrtPriceX96 管“格子里的高精度真实价格”。

Q46 Lending Protocol 的 Health Factor 是什么?

难度 ★★★ | 高频度 ★★★★★ | 岗位 DeFi/后端

面试官意图 借贷协议风险引擎核心题。

30 秒回答

Health Factor 把抵押品按 liquidation threshold 加权后，与总借款价值比较。以 Aave 的概念公式：`HF = CollateralValue * WeightedLiquidationThreshold / BorrowValue`；HF 低于 1 时仓位进入可清算区域。

深入解析

Health factor 是动态风险指标，会随 oracle price、借贷利息、抵押变化和治理参数变化。不是所有 `HF=1.2` 都同样安全，高波动/低相关资产需要更大 buffer。

工程系统需要监控 position、oracle update 和 liquidation eligibility；前端展示的“安全度”不是静态字段，而是实时派生值。

$$HF = \Sigma(collateral_i * price_i * LT_i) / totalBorrowValue$$

HF 越接近 1，清算风险越高

必须说出的关键点

- weighted liquidation threshold
- HF < 1 => liquidation eligibility in Aave model
- dynamic with prices/debt accrual
- risk buffer depends on volatility/correlation

高频追问

- LTV 与 liquidation threshold 有何区别？借款上限 vs 清算触发阈值。
- 为什么利息增长也会降低 HF？BorrowValue 上升。
- 清算是否一定 100% debt？具体 close factor 依协议规则。

常见误区 / Gotchas

- 把 HF 当“抵押率百分比”而不写分子分母。
- 假设稳定币抵押永远无波动。

记忆锚点 Health Factor 是“按清算阈值折算后的有效抵押 / 当前债务”。

参考：R10 Aave Health Factor

Q47 为什么 DeFi Lending 通常需要 Over-collateralization?

难度 ★★ | 高频度 ★★★★☆ | 岗位 DeFi

面试官意图 从无身份信用执行推导抵押模型。

30 秒回答

Permissionless lending 通常无法依赖现实身份和法律追偿，因此协议要在链上用可立即处置的抵押品覆盖债务，并在抵押价值下降到危险区前触发清算。

深入解析

过度抵押不是“资本低效的错误”，而是无信用身份环境下换取 solvency 的机制。是否能降低抵押率取决于资产波动、oracle 质量、清算流动性、borrower identity 或其他信用担保。

协议真正关心的是在价格跳跃和拥堵情况下是否还能以足够价值卖出 collateral 偿还 debt。

必须说出的关键点

- no legal recourse -> onchain collateral
- liquidation buffer absorbs volatility
- oracle/liquidity determine safe LTV
- bad debt is failure mode

高频追问

- 为什么现实世界资产借贷可用不同模型？存在 offchain enforcement。
- 抵押率设置太高/太低分别有什么代价？资本效率 vs insolvency risk。
- stable collateral 为什么也要 haircut？depeg/correlation risk。

常见误区 / Gotchas

- “因为 DeFi 不相信用户”这种空泛说法。
- 只看价格波动，不看清算深度。

记忆锚点 无信用身份时，协议只能让“可自动处置的资产”替代“承诺以后还钱”。

Q48 Liquidation 的经济机制是什么?

难度 ★★★ | 高频度 ★★★★★ | 岗位 DeFi/安全/后端

面试官意图 考 solvency、incentive 和执行竞争。

30 秒回答

当仓位低于协议安全阈值，permissionless liquidator 可以偿还一部分或全部债务，并获得对应抵押品加 liquidation bonus。Bonus 用来补偿 gas、价格风险和竞争成本，使外部参与者有动力及时恢复协议健康。

深入解析

清算参数必须平衡：bonus 太小，市场剧烈波动时没人愿意清算；太大，则 borrower 损失高且可能制造可利用边界。还要设置 close factor、dust 规则和 oracle safeguards。

极端行情的真正风险是“可清算但执行不掉”：网络拥堵、oracle 延迟、DEX 深度不足都可能让 bad debt 累积。

必须说出的关键点

- permissionless execution
- liquidation incentive
- close factor/dust
- oracle + market liquidity + chain liveness

高频追问

- 清算机器人为什么竞争激烈？奖励属于先执行成功者。
- 如果 collateral 没有市场流动性怎么办？协议可能产生坏账。
- 为什么 oracle update 与 liquidation bot 延迟很重要？窗口决定风险。

常见误区 / Gotchas

- 把 liquidation 当惩罚机制而不是 solvency mechanism。
- 认为 HF<1 后协议自动瞬间清算，无需执行者。

记忆锚点 清算是把协议偿付能力外包给一个有奖励的公开执行市场。

参考：R10 Aave Health Factor

Q49 Stablecoin 有哪些主要设计与 Failure Modes?

难度 ★★★ | 高频度 ★★★★☆ | 岗位 DeFi/协议

面试官意图 不是考项目名，而是 peg mechanism 与资产负债表。

30 秒回答

可按主要 backing 分为法币储备型、链上超额抵押型、以及更多依赖内生资产/算法激励的设计。分析任何 stablecoin 都应问：1 unit 为什么应该值目标价格、谁能赎回、资产在哪里、压力下谁吸收损失。

深入解析

法币储备型主要有托管、银行、赎回和监管风险；超额抵押型有 oracle、清算和 collateral volatility 风险；内生/算法模型尤其要警惕 reflexive death spiral：支撑资产价格与 stablecoin 信心同时下跌。

Peg 不是“价格永远等于 1”，而是套利/赎回机制使偏离后有经济力量把价格拉回。分析时画清楚 mint、redeem、liquidation、reserve asset 和最后损失承担者。

必须说出的关键点

- peg mechanism
- redemption path
- reserve quality/liquidity
- loss absorber
- reflexivity

高频追问

- 为什么高流动性 redemption 比口头“100% backed”更重要？压力时要能兑现。
- 稳定币作为 collateral 时如何处理 depeg？动态 oracle 和 haircut。
- 跨链 wrapped stablecoin 多了什么风险？bridge risk。

常见误区 / Gotchas

- 把所有 stablecoin 都叫“算法稳定币”。
- 只看 reserve 数字，不看 redemption/legal control。

记忆锚点 问 stablecoin 四句话：拿什么支撑、怎么赎回、谁能套利、崩的时候谁先亏。

Q50 设计 DeFi Protocol 时最重要的 Invariant 是什么?

难度 ★★★★★ | 高频度 ★★★★★ | 岗位 Protocol/Senior

面试官意图 从功能需求升级到可形式化的资产安全性质。

30 秒回答

没有一个适合所有协议的 invariant。AMM、vault、lending、stablecoin 都应先写自己的守恒或偿付条件，例如 vault 中总 shares 与 totalAssets 的换算必须一致，lending 中可兑现资产不能系统性小于负债。

深入解析

Invariant 是“任意合法操作序列之后都应该成立的性质”。它比单元测试里的具体输入输出更强，因为攻击者不会按产品 happy path 调函数。

常见类别包括 conservation（资产守恒）、solvency（资产覆盖负债）、authorization（未授权者不能移动资产）、monotonicity（某些指数只能单调变化）、boundedness（费率/价格/份额不能越界）。

必须说出的关键点

- write invariants before code
- asset conservation
- share/accounting consistency
- solvency under adversarial ordering
- fuzz stateful sequences

高频追问

- ERC4626 vault 的核心 invariant 有哪些？shares/assets conversion 和 total accounting。
- 手续费加入后 invariant 怎么变化？写成带 fee 的守恒。
- 如何验证 invariant？Foundry invariant fuzz / symbolic tools / differential models。

常见误区 / Gotchas

- 把“函数不 revert”当正确性。
- 只有 unit tests，没有随机操作序列。

记忆锚点 高级协议开发不是“实现 20 个函数”，而是“让 5 个关键不变量在任意调用顺序下永远成立”。

CHAPTER 06

Layer2、Rollup、ZK 与 Bridge

本章目标 扩容题要按 execution、data availability、settlement、proof、sequencing 五层拆解。只说“L2 更快更便宜”远远不够。

本章题目

Q51 为什么 Ethereum 需要 Layer2?
Q52 Optimistic Rollup 的工作原理是什么?
Q53 ZK Rollup 与 Optimistic Rollup 的核心区别?
Q54 EIP-4844 为什么对 Rollup 重要?
Q55 Data Availability 到底是什么意思?
Q56 Sequencer 是什么?中心化 Sequencer 有哪些风险?
Q57 Validium 与 ZK Rollup 有什么区别?
Q58 为什么 Bridge 是 Web3 高风险组件?
Q59 ZK Proof 的 Completeness、Soundness、Zero-Knowledge 分别是什么?
Q60 设计一个 Rollup 需要哪些核心组件?

Q51 为什么 Ethereum 需要 Layer2?

难度 ★★ | 高频度 ★★★★★ | 岗位 全岗位

面试官意图 从全节点重复执行的安全模型推导扩容瓶颈。

30 秒回答

L1 要求大量节点验证同一状态转换，因此不能无限提高每块计算和数据量。Rollup 把大量 execution 移到 L2，再把足够的数据和状态证明/争议机制锚定到 Ethereum，从而摊薄每笔交易的 L1 成本。

深入解析

“把计算搬走”只是第一步。要继承 L1 安全，其他参与者仍必须能获得交易数据、验证状态转换，并在 sequencer 作恶或离线时有退出/强制包含路径。

所以评价 L2 不能只看 TPS，要看 DA、proof system、bridge、sequencer、upgrade keys、forced inclusion 和 L1 settlement。

必须说出的关键点

- execution scales off L1
- security depends on DA + verification + settlement
- cost amortization via batching
- L2 has additional trust/liveness assumptions

高频追问

- Sidechain 与 Rollup 最大区别？是否把安全/数据验证锚定到 L1。
- 为什么只把 state root 发 L1 不够？没有数据无法重建状态。
- L2 出块快是否等于 finality 快？不一定。

常见误区 / Gotchas

- 把任何“跑在 Ethereum 旁边的链”都叫 L2。
- 只比较 TPS 和 gas。

记忆锚点 Rollup 的价值是“把执行批量化，同时保留足够证据让 L1 能裁决”。

Q52 Optimistic Rollup 的工作原理是什么?

难度 ★★★ | 高频度 ★★★★★ | 岗位 L2/协议

面试官意图 考假设有效 + challenge 的安全模型。

30 秒回答

Optimistic Rollup 默认接受 L2 提交的状态结果，但保留挑战机制：如果有人证明某个状态转换无效，协议可以纠正。为了让挑战可行，L2 数据必须可用，且挑战者要有时间观察和提交证明。

深入解析

“optimistic” 不是“完全不验证”。节点仍执行 L2 交易，L1 contract 负责 settlement/challenge 规则。历史实现常有 challenge period，因此原生 bridge withdrawal 可能需要等待。

系统还要解决 sequencer censorship：即使 sequencer 不愿收交易，用户最终应有 L1 forced inclusion 或 escape mechanism。

必须说出的关键点

- assume valid until challenged
- data must be available
- fraud/dispute mechanism
- challenge window affects withdrawals

高频追问

- 如果没人监控会怎样？安全依赖至少一个诚实观察者/验证参与者。
- 为什么 L2 full node 仍需执行交易？要验证 state。
- fault proof 与 fraud proof 术语差异？看具体实现。

常见误区 / Gotchas

- 说“Optimistic Rollup 不做验证”。
- 忽略 upgrade multisig 可能比 fault proof 更强的实际信任假设。

记忆锚点 Optimistic 的安全承诺是：错误可以被公开数据重放并在窗口内挑战。

Q53 ZK Rollup 与 Optimistic Rollup 的核心区别?

难度 ★★★ | 高频度 ★★★★★ | 岗位 L2/ZK

面试官意图 比较 validity vs dispute，不是“ZK 更隐私”。

30 秒回答

ZK Rollup 为状态转换生成 validity proof，由 L1 verifier 验证“这批计算满足电路/VM 规则”；Optimistic Rollup 则先接受结果并允许后续 challenge。ZK Rollup 的“ZK”在 Rollup 场景重点是 succinct validity，并不必然提供交易隐私。

深入解析

Validity proof 可以让 L1 不重做整批计算，只验证一个短证明。代价在 prover 成本、电路/zkVM 复杂度、证明延迟和工程兼容性。

两类系统都仍需要 data availability；一个正确 state root 的 proof 并不能让用户在数据被隐藏时恢复自己的状态和退出。

必须说出的关键点

- validity proof vs challenge
- ZK != privacy by default
- both need DA
- prover cost vs challenge latency

高频追问

- STARK 与 SNARK 大致 trade-off？proof size、trusted setup、hash/field assumptions 等。
- ZK proof 为什么可以很小？succinct proof system。
- 证明快是否意味着 L2 finality 等同 L1 finality？仍要等 L1 inclusion/finality。

常见误区 / Gotchas

- “ZK Rollup 因为 zero knowledge 所以交易私密”。
- 忽略 proof generation latency。

记忆锚点 Optimistic 说“错了你来挑战”；ZK Rollup 说“我先给你一个可验证的正确性证明”。

Q54 EIP-4844 为什么对 Rollup 重要?

难度 ★★★★ | 高频度 ★★★★★ | 岗位 L2/Ethereum

面试官意图 2026 Ethereum 扩容必考更新。

30 秒回答

EIP-4844 引入 blob-carrying transaction，为 Rollup 提供独立于普通 execution calldata 的大数据承载空间。Blob 内容不能被 EVM 直接读取，但其 commitment 可被执行层访问，并使用独立 blob fee market。

深入解析

Rollup 成本里很大一部分来自把交易数据发布到 L1 以保证 data availability。Blob 正是为这种“需要共识层暂时保留、但 EVM 不需长期执行读取”的数据设计。

EIP-4844 是通向更完整 data sharding/DAS 的过渡架构。面试中最重要的区别：blob 不是 cheap calldata；它有独立生命周期、KZG commitment 和 fee accounting。

必须说出的关键点

- blob data is not EVM-readable
- commitment/versioned hash is available
- separate blob gas market
- rollup DA cost reduction

高频追问

- 为什么 blobs 可以被较早删除？执行状态不依赖永久 EVM 可读内容，历史归档由生态层承担。
- blob transaction 能创建 contract 吗？EIP-4844 `to` 必须非空。
- calldata 是否因此没用了？仍用于 EVM 必须直接读取的数据。

常见误区 / Gotchas

- 把 blob 当“更大的 calldata”。
- 认为 blobs 永久存储在 Ethereum execution state。

记忆锚点 Blob 是给 Rollup 的“数据可用性带宽”，不是给合约当普通 storage 用的。

参考：R2 EIP-4844

Q55 Data Availability 到底是什么意思?

难度 ★★★★ | 高频度 ★★★★★ | 岗位 L2/协议

面试官意图 Rollup 安全的核心概念。

30 秒回答

Data Availability 指验证者/用户是否能获得重建状态转换所需的数据。只有 state root 或 proof 而拿不到交易/状态差分数据，用户可能无法独立验证余额、构造退出或继续链。

深入解析

DA 与 data correctness 不同：数据可能“可获得但内容无效”，这由执行验证/proof 发现；也可能“root/proof 看起来合法但底层数据被 withheld”，此时用户无法重建。

Rollup 把 data 放 L1 的主要目的就是继承 Ethereum 的 DA 保证。Validium 则把 DA 放到链外委员会/专门 DA 层，成本更低但增加信任假设。

必须说出的关键点

- availability != validity
- users need data to reconstruct/exit
- L1 blobs provide Ethereum-backed DA
- offchain DA changes trust model

高频追问

- DA sampling 解决什么？在不下载全部数据时概率检查数据是否可用。
- 为什么全节点下载全部 blob 不等于最终 danksharding？未来目标包括采样扩容。
- Celestia/其他 DA 层与 Ethereum DA 如何比较？看安全集合与 bridge assumptions。

常见误区 / Gotchas

- 把 DA 理解为“数据库在线率”。
- 只要有 validity proof 就认为 data withholding 无害。

记忆锚点 Validity 回答“算得对不对”；Availability 回答“大家拿不拿得到用于自己验证和退出的数据”。

参考：R2 EIP-4844

Q56 Sequencer 是什么？中心化 Sequencer 有哪些风险？

难度 ★★★ | 高频度 ★★★★★ | 岗位 L2/后端

面试官意图 L2 transaction lifecycle 与 censorship/liveness 题。

30 秒回答

Sequencer 接收 L2 交易、决定排序并快速给出执行结果，再把 batch 提交到更底层结算/DA 层。中心化 sequencer 可以提供低延迟 UX，但带来 censorship、downtime、ordering/MEV 和单点控制风险。

深入解析

安全设计要区分 safety 与 liveness：sequencer 通常不应该能凭空伪造最终有效状态（proof/challenge 约束），但可能拒绝服务或延迟用户交易。

因此成熟 Rollup 会设计 forced inclusion、L1 queue、escape hatch、sequencer rotation 或 shared sequencing 等机制。

必须说出的关键点

- ordering power
- censorship/liveness risk
- MEV control
- forced inclusion is critical escape path

高频追问

- sequencer 关闭时用户资金是否安全？取决于 L1 bridge/forced withdrawal 设计。
- decentralized sequencer 解决全部问题吗？还要处理共识和跨域 MEV。
- soft confirmation 是什么？sequencer 先给的非 L1 final assurance。

常见误区 / Gotchas

- 把中心化 sequencer 直接等同于能盗走所有资金。
- 只看“是否去中心化”，不区分 safety 和 liveness 权限。

记忆锚点 Sequencer 最强的权力通常是“谁先执行、谁暂时不执行”；真正资产安全还取决于 settlement proof 和 bridge。

Q57 Validium 与 ZK Rollup 有什么区别?

难度 ★★★★ | 高频度 ★★★★☆ | 岗位 L2/ZK

面试官意图 考 proof 与 DA 两个维度是否能独立拆开。

30 秒回答

两者都可以用 validity proof 证明状态转换正确，主要区别在数据可用性：ZK Rollup 把必要数据发布到 Ethereum/L1；Validium 把数据放在链外或另一个 DA 机制。

深入解析

把 DA 移出 L1 可显著降低成本和提高吞吐，但如果 DA provider withholding，用户即使相信 proof 表明“状态转换合法”，也可能拿不到自己的状态数据完成退出。

因此 Validium 不是“更差的 ZK Rollup”，而是不同 cost/trust trade-off。某些业务可以接受委员会/专门 DA 的假设。

必须说出的关键点

- same validity dimension, different DA
- cheaper data, stronger availability assumption
- proof correctness does not solve withholding
- classify systems on multiple axes

高频追问

- Volition 是什么？允许用户/交易选择不同 DA mode 的混合设计。
- Validium operator 能否随意偷资产？如果 proof 电路正确，不能提交无效 state，但可产生 DA liveness 风险。
- 如何评价 DA committee？阈值、成员、退出机制。

常见误区 / Gotchas

- 把 Validium 叫“没有 ZK proof”。
- 只按营销名分类，而不问数据到底放哪里。

记忆锚点 Proof 决定“算得对不对”，DA 决定“数据在不在大家手里”；Validium 只改变后者。

Q58 为什么 Bridge 是 Web3 高风险组件?

难度 ★★★ | 高频度 ★★★★★ | 岗位 跨链/安全

面试官意图 跨域状态验证与 custody 模型核心。

30 秒回答

Bridge 必须让链 B 相信“链 A 上某件事确实发生并且足够最终”。这个信任可以来自 multisig/validator set、light client、optimistic proof、ZK proof 等。任何验证错误都可能让 B 铸出没有真实 backing 的资产。

深入解析

Bridge 常同时管理大量集中流动性，且跨链状态机复杂：source lock/burn、message relay、finality、destination mint/unlock、replay protection、rate limits 都必须正确。

审计时要先问“谁能伪造跨链消息？”、“source reorg 怎么办？”、“destination 执行失败如何重试？”、“validator keys 被攻破最大损失是多少？”

必须说出的关键点

- cross-domain state verification
- message authenticity + finality
- lock/mint or burn/mint supply invariant
- replay + retry + reorg handling

高频追问

- Light-client bridge 为什么信任最小？目标链验证源链共识证明。
- 为什么 multisig bridge 攻击面大？阈值密钥是高价值单点集合。
- rate limit 有什么用？限制单次故障 blast radius。

常见误区 / Gotchas

- 只审 destination token contract，不审 relayer/verifier。
- 认为“跨链消息只执行一次”自然成立，未设计 nonce/message id。

记忆锚点 Bridge 的本质不是“搬币”，而是“在另一条链上可信地证明并执行一个远端事实”。

Q59 ZK Proof 的 Completeness、Soundness、Zero-Knowledge 分别是什么?

难度 ★★★★ | 高频度 ★★★★☆ | 岗位 ZK/研究

面试官意图 基础密码学证明系统题。

30 秒回答

Completeness：真实 statement + 正确 witness 时诚实 prover 能让 verifier 接受；Soundness：错误 statement 很难被恶意 prover 骗过；Zero-knowledge：verifier 除了 statement 成立外，不应获得 witness 的额外信息。

深入解析

Rollup 最核心通常是 soundness/validity 和 succinct verification；是否需要 zero-knowledge/privacy 取决于具体系统。一个 proof 可以是 succinct validity proof，但公开全部交易数据。

还可继续谈 knowledge soundness、trusted setup、proof size、verification cost、prover time 等工程指标。

必须说出的关键点

- completeness = honest proof works
- soundness = false claim cannot pass
- zero-knowledge = hides witness
- succinctness is another property

高频追问

- 为什么 “ZK Rollup” 不必隐私？proof 可验证公开状态转换。
- trusted setup 被破坏会影响什么？取决于 proof system，可能损害 soundness。
- STARK 为什么常不需要 trusted setup？基于不同证明构造。

常见误区 / Gotchas

- 把 zero-knowledge 等同于 encryption。
- 认为所有 ZK proof 都必须极小。

记忆锚点 Completeness 保证好人能证明，Soundness 防坏人伪造，Zero-knowledge 防验证者偷看 witness。

Q60 设计一个 Rollup 需要哪些核心组件?

难度 ★★★★★ | 高频度 ★★★★★ | 岗位 L2/Staff

面试官意图 系统设计终极题之一。

30 秒回答

至少拆成 execution engine、sequencer、state database、batcher、DA publisher、L1 bridge/contracts、proof/challenge system、node/RPC、withdrawal/forced inclusion、indexer/monitoring 和 governance/upgrades。

深入解析

先画数据流：用户 tx -> sequencer -> L2 execution -> batch -> DA/L1 commitment -> proof/challenge -> L1 settlement。再为每条边写 failure mode。

Staff 级回答必须标出 trust boundary：sequencer 能做什么、prover 能做什么、upgrade multisig 能做什么、L1 contract 是否 pauseable、数据 unavailable 时用户如何 permissionlessly exit。

必须说出的关键点

- execution
- ordering/sequencing
- DA publishing
- settlement/verifier
- bridge/withdrawal
- forced inclusion
- node/indexer/monitoring
- governance and upgrade keys

高频追问

- sequencer 和 proposer 是否同一个概念？依架构而定。
- proof system 挂了怎么办？系统可能停止 finalization，需 emergency/escape 设计。
- 如何逐步 decentralize？按 sequencer/prover/governance/DA 分轴。

常见误区 / Gotchas

- 列组件但没有 trust model。
- 只设计 happy path，不说 sequencer/prover/L1 RPC 故障。

记忆锚点 Rollup 系统设计要同时画两张图：交易数据流图 + 权限/信任边界图。

CHAPTER 07

钱包、签名与 Account Abstraction

本章目标 钱包题的本质是授权系统设计：谁能签、签什么、签名能否重放、谁付 gas、账户如何恢复和限制权限。

本章题目

Q61 EOA 与 Smart Contract Account 有什么区别?
Q62 ecrecover 做什么？有哪些安全注意事项?
Q63 什么是 Signature Replay Attack?
Q64 EIP-712 解决什么问题?
Q65 ERC-2612 Permit 是什么?
Q66 Account Abstraction 解决哪些 UX 与安全问题?
Q67 ERC-4337 的 UserOperation 与 EntryPoint 流程是什么?
Q68 ERC-4337 Paymaster 是做什么的?
Q69 EIP-7702 与 ERC-4337 是什么关系?
Q70 如何设计一个生产级 Smart Wallet?

Q61 EOA 与 Smart Contract Account 有什么区别?

难度 ★★ | 高频度 ★★★★★ | 岗位 钱包/AA/全栈

面试官意图 建立账户抽象的起点。

30 秒回答

传统 EOA 由协议固定的私钥签名规则控制，没有自身 EVM code；Smart Contract Account 的行为由合约代码定义，可以实现 multisig、session key、recovery、spending policy 等自定义验证与执行逻辑。

深入解析

历史 Ethereum transaction 只能由 EOA 发起，因此传统 smart wallet 仍需某个 EOA 付 gas 并触发合约。ERC-4337 通过 UserOperation/bundler 绕开这个 UX 限制；EIP-7702 又让 EOA 可以委托执行合约代码。

讨论账户时要区分“谁拥有地址”“谁能发 transaction”“谁验证授权”“谁付 gas”四个问题，它们在 AA 世界里不再绑定在一个私钥上。

必须说出的关键点

- EOA protocol-fixed validation
- smart account programmable validation
- gas payer can be abstracted
- recovery/session/batch are account logic

高频追问

- contract account 能否自己主动发 L1 transaction？传统模型不能，需要外部 transaction 触发。
- EOA 有 code 后还是 EOA 吗？EIP-7702 引入更复杂语义，不能沿用旧二分法。
- multisig 为什么通常是 smart account？签名阈值在合约逻辑中验证。

常见误区 / Gotchas

- 把 EOA 与“用户”画等号；一个用户可能控制多个账户。
- 把 smart account 当作“更安全”而不讨论代码/升级风险。

记忆锚点 AA 的核心是把“账户如何证明这次操作被授权”从协议固定逻辑变成可编程逻辑。

参考：R11 ERC-4337, R3 EIP-7702

Q62 ecrecover 做什么？有哪些安全注意事项？

难度 ★★★ | 高频度 ★★★★☆ | 岗位 Solidity/钱包/安全

面试官意图 `ecrecover` 是 permit、订单签名和自定义授权的底层高频工具。

30 秒回答

`ecrecover(hash, v, r, s)` 根据 secp256k1 ECDSA 签名恢复 signer address。合约可以把恢复地址与期望 owner 比较，从而验证链下签名授权。

深入解析

真正难点不在调用 opcode，而在 message digest 构造：必须绑定 action、arguments、contract、chain、nonce、deadline 等，否则一个有效签名可能在另一个上下文重放。

还应使用标准库处理签名 malleability、EIP-2098 compact signature、ERC-1271 contract signature 等兼容问题，不建议业务合约手写所有边界。

必须说出的关键点

- recover signer from digest+signature
- digest design is security-critical
- nonce/domain prevent replay
- support contract signatures when needed

高频追问

- 为什么 `abi.encodePacked` 拼签名消息可能危险？动态类型存在编码歧义。
- ERC-1271 解决什么？smart contract signature validation。
- v/r/s 里的 low-s 规则为什么存在？减少 malleability。

常见误区 / Gotchas

- 直接 `keccak256(abi.encodePacked(to, amount))` 就当完整签名方案。
- 只比较 signer，不消费 nonce。

记忆锚点 签名验证难点不是“恢复地址”，而是“保证这个地址只授权了这一次、这个动作、这个上下文”。

Q63 什么是 Signature Replay Attack?

难度 ★★★ | 高频度 ★★★★★ | 岗位 钱包/安全/DeFi

面试官意图 授权协议必考。

30 秒回答

Replay 是攻击者把同一个有效签名再次提交，或换一个链/合约/上下文提交，而验证逻辑仍接受。根因是签名消息没有包含唯一性和 domain binding，或者 nonce 没被正确消费。

深入解析

典型防护字段：nonce 保证单次/顺序，deadline 限制时间，chainId 防跨链，verifyingContract 防跨合约，action/type 防把一种签名解释成另一种操作。EIP-712 提供标准化 typed-domain 结构。

Nonce 设计也有 trade-off：单调全局 nonce 简单但阻塞并发；bitmap/unordered nonce 允许并发但实现更复杂。ERC-4337 本身支持更灵活的 nonce 结构。

必须说出的关键点

- nonce
- chain/domain binding
- deadline
- consume-before-external-call when appropriate

高频追问

- 为什么只加 chainId 仍可能跨两个同地址合约重放？还要 verifyingContract。
- permit nonce 应什么时候递增？验证成功并执行授权的原子流程内。
- 跨链 intentionally replayable 签名如何设计？domain 中显式表达允许范围。

常见误区 / Gotchas

- 签名里有 timestamp 就认为防重放。timestamp 不唯一。
- nonce 存在但失败交易也错误消耗/不消耗，导致逻辑混乱。

记忆锚点 Replay 防护就是回答：这份授权在哪里有效、对什么有效、能用几次、能用到什么时候。

Q64 EIP-712 解决什么问题?

难度 ★★★ | 高频度 ★★★★★ | 岗位 钱包/前端/Solidity

面试官意图 typed data 与 domain separation 核心。

30 秒回答

EIP-712 为结构化数据签名定义 type hash、struct hash 和 domain separator，让钱包能展示“你正在签什么字段”，同时把签名绑定到应用/链/合约语境，降低盲签与 replay 风险。

深入解析

核心思想是把消息从“模糊 bytes”变成 schema + values。Digest 通常由 EIP-191 prefix、domain separator 和 structHash 组合。

安全实现要确保前端展示字段与合约实际验证字段完全一致；类型版本、chainId、verifyingContract 任何不匹配都会导致签名无效或 domain 错误。

必须说出的关键点

- typed structured data
- domain separator
- human-readable wallet display
- replay resistance when domain+nonce used correctly

高频追问

- EIP-712 自己是否自动提供 nonce？不提供，业务结构必须定义。
- domain version 有什么用？协议升级时隔离旧签名。
- 为什么不能签 JSON 字符串？序列化顺序/歧义与 UX 都差。

常见误区 / Gotchas

- 认为 712 自动防所有 replay。
- 前端和合约使用不同 type string。

记忆锚点 EIP-712 把“签一串看不懂的 bytes”变成“签一个有类型、有域、有字段的授权对象”。

Q65 ERC-2612 Permit 是什么?

难度 ★★★ | 高频度 ★★★★☆ | 岗位 DeFi/钱包

面试官意图 典型 meta-authorization 题。

30 秒回答

Permit 允许 token holder 在链下签署 allowance 授权，任何人把签名提交给 token contract 即可设置 allowance，因此用户无需先单独发送 approve transaction。

深入解析

Permit 不是“免 gas”，而是把 owner 的 approve transaction 替换成签名；最终 permit + downstream action 仍由某个 relayer/spender transaction 执行。它改善“先 approve 再 swap”的两步 UX。

安全关键仍是 EIP-712 domain、nonce、deadline 和 spender/amount 字段。不同 token 的 permit 实现可能存在兼容差异，前端不能盲目假设都支持。

必须说出的关键点

- offchain signed approval
- onchain relayer submits
- nonce/deadline/domain
- reduces one user transaction

高频追问

- Permit2 与 ERC-2612 思路有何不同？外部统一授权层与更丰富权限模型。
- 为什么 permit 交易可以由别人提交？签名已经证明 holder 授权。
- front-running permit 会盗币吗？如果 action 设计正确，抢先提交同授权不应改变 recipient/amount 语义。

常见误区 / Gotchas

- 说“permit 不需要任何链上交易”。
- 忽略签名 nonce。

记忆锚点 Permit 把“我亲自发 approve 交易”变成“我签一个别人也能代交的 approve 授权”。

Q66 Account Abstraction 解决哪些 UX 与安全问题?

难度 ★★★ | 高频度 ★★★★★ | 岗位 钱包/产品/协议

面试官意图 不应只回答“gasless”。

30 秒回答

AA 的目标是把账户验证和执行变成可编程能力，从而支持 social recovery、multisig、session keys、spending limits、batching、gas sponsorship、不同签名算法和更细粒度权限。

深入解析

传统 EOA 把“拥有私钥 = 拥有全部账户能力”绑定得很死。Smart account 可以把权限拆分：游戏 session key 只能操作特定合约且每天有限额；guardian 只能恢复而不能转账；paymaster 可赞助特定操作的 gas。

代价是系统复杂度显著上升：validation DoS、module/plugin 安全、upgrade、bundler/paymaster 兼容、recovery governance 都是新的攻击面。

必须说出的关键点

- programmable validation
- programmable execution
- gas abstraction
- least-privilege keys/recovery
- complexity and module risk

高频追问

- 为什么 session key 比把主私钥给 dApp 好？权限更小且可过期。
- AA 是否意味着没有私钥？不一定，只是验证方式不固定。
- smart wallet recovery 的最大风险？guardian collusion / social engineering / upgrade authority。

常见误区 / Gotchas

- 把 AA 简化成“免 gas”。
- 只谈 UX 不谈新的信任边界。

记忆锚点 AA 的本质是把账户从“一把万能钥匙”改造成“可编程权限系统”。

Q67 ERC-4337 的 UserOperation 与 EntryPoint 流程是什么?

难度 ★★★★ | 高频度 ★★★★★ | 岗位 AA/钱包/协议

面试官意图 2026 账户抽象核心题。

30 秒回答

ERC-4337 定义 UserOperation 作为高层 pseudo-transaction。用户把 UserOp 发送到专用 mempool，bundler 收集后调用 EntryPoint `handleOps`，EntryPoint 验证 smart account/paymaster 并执行实际操作。

深入解析

它不要求 Ethereum consensus layer 改 transaction validation，而是在合约和 P2P 基础设施层建立新的操作管道。核心角色包括 smart account、EntryPoint、bundler、factory、paymaster，可选 signature aggregator。

Bundler 在打包前必须 simulation validation，避免恶意 account/paymaster 让 bundler 无偿烧 gas。EntryPoint 负责统一 gas accounting 与执行边界。

必须说出的关键点

- UserOperation != native Ethereum transaction
- bundler submits handleOps tx
- EntryPoint central contract abstraction
- simulation rules mitigate DoS/griefing

高频追问

- 谁支付 bundler gas? account deposit 或 paymaster，最终由 EntryPoint accounting。
- 首次账户未部署怎么办? factory/init code 机制。
- 为什么 UserOp 需要独立 mempool rules? 验证逻辑可编程，DoS 风险比 native tx 高。

常见误区 / Gotchas

- 把 bundler 当可信中心化服务器；协议目标是可竞争 bundlers。
- 认为 4337 修改了 EVM transaction type。

记忆锚点 4337 把“用户直接发 Ethereum tx”拆成“用户发 UserOp，bundler 代打包，EntryPoint 统一验证与结算”。

参考：R11 ERC-4337

Q68 ERC-4337 Paymaster 是做什么的?

难度 ★★★★ | 高频度 ★★★★☆ | 岗位 AA/钱包

面试官意图 gas sponsorship 与 griefing 题。

30 秒回答

Paymaster 可以在满足自定义条件时替用户承担 UserOperation 的 gas 成本，例如应用为新用户赞助手续费，或允许用其他资产/商业逻辑结算。

深入解析

因为 paymaster 可能被恶意 UserOp 反复触发验证却不产生有效业务，4337 对 validation、stake/deposit 和 simulation 有严格约束，目标是防止 bundler/paymaster 被 DoS 或 gas griefing。

业务上不要把 paymaster 设计成“任何请求都免费”。常见策略会绑定 target、selector、额度、用户资格、签名 coupon、时间窗和每日预算。

必须说出的关键点

- gas sponsorship policy
- EntryPoint deposit/stake
- validation must be bounded/predictable
- anti-abuse quotas/auth

高频追问

- paymaster 能否改变用户 callData？通常验证/赞助，不应偷偷改变账户操作。
- 为什么 sponsor server 签 coupon 比 onchain allowlist 更灵活？可动态风控。
- paymaster 下线会怎样？UserOp 可能无法被赞助，账户应有自付路径。

常见误区 / Gotchas

- 无限免费 gas，没有预算/资格限制。
- 把 paymaster 账户当作用户资产 custodian。

记忆锚点 Paymaster 是“可编程的 gas 赞助商”，不是“凭空让交易没有成本”。

参考：R11 ERC-4337

Q69 EIP-7702 与 ERC-4337 是什么关系?

难度 ★★★★★ | 高频度 ★★★★★ | 岗位 AA/高级 Ethereum

面试官意图 当前账户抽象的辨析题。

30 秒回答

EIP-7702 是协议级新 transaction type，使 EOA 可通过 authorization 设置 delegation indicator，让调用执行委托合约代码，从而获得 batching、sponsorship、权限降级等能力；ERC-4337 则是无需 consensus 改动的 UserOperation/EntryPoint 基础设施。

深入解析

二者不是简单替代。7702 让“现有 EOA 地址”获得智能账户式代码能力，4337 提供 bundler、paymaster、UserOp mempool 和统一 EntryPoint 生态。当前 ERC-4337 规范已明确 `Requires EIP-7702` 并包含 EIP-7702 delegated smart account 支持。

安全上 7702 带来新的 EOA storage/code 语义、初始化 front-running、delegation target 安全和 sponsored relayer griefing 等问题。旧代码依赖 `tx.origin`/`code.length==0` 判断 EOA 的假设更脆弱。

必须说出的关键点

- 7702: protocol transaction + EOA code delegation
- 4337: alt mempool + UserOp + EntryPoint
- can be complementary
- legacy EOA assumptions break

高频追问

- 7702 delegation 是复制 code 还是指向目标？写入 delegation indicator 并执行目标 code。
- 授权能跨链吗？authorization tuple chain_id 可约束语义。
- 为什么 4337 仍有价值？完整 bundler/paymaster/mempool/account ecosystem。

常见误区 / Gotchas

- 说“7702 已经淘汰 4337”。
- 继续用 `address.code.length==0` 当人类用户判断。

记忆锚点 7702 改“EOA 能不能像合约一样执行”；4337 改“用户操作怎样被验证、赞助和打包进链”。

参考：R3 EIP-7702, R11 ERC-4337

Q70 如何设计一个生产级 Smart Wallet?

难度 ★★★★★ | 高频度 ★★★★★ | 岗位 钱包/Staff

面试官意图 系统设计：身份、执行、恢复、插件、gas 与升级的完整边界。

30 秒回答

至少拆成 authentication、nonce/replay、execution/batch、recovery、session permissions、spending policy、gas payment、upgrade/module system 和 emergency controls。每个模块都要写出“谁能改变它”和“被攻破最大损失”。

深入解析

签名层需要支持主密钥、硬件签名、ERC-1271 或多签；权限层需要 session key 的 target/selector/value/time limits；恢复层需要 guardian threshold 和 delay；执行层要避免 arbitrary delegatecall plugin 获得无限权限。

如果支持 4337/7702，还要定义 bundler/paymaster fallback、EntryPoint compatibility、counterfactual deployment 和迁移策略。最重要的是用户在基础设施故障时仍有自救路径。

必须说出的关键点

- auth != execution != recovery
- module permissions least-privilege
- nonce domains for concurrency
- upgrade/recovery are highest-risk capabilities
- fallback path without proprietary infra

高频追问

- 插件系统如何防恶意 module？capability sandbox/allowlist/审计/权限声明。
- social recovery 如何防 guardian 突然接管？threshold + delay + cancel path。
- 账户升级后旧签名怎么失效？versioned domain/nonce migration。

常见误区 / Gotchas

- 功能越多越好，结果形成万能 delegatecall module。
- 依赖唯一 bundler/paymaster 服务，没有自付/迁移方案。

记忆锚点 Smart Wallet 不是“一个多签合约”，而是一套把认证、权限、执行、恢复和 gas 分离的安全操作系统。

CHAPTER 08

Oracle、MEV 与 Web3 基础设施

本章目标 合约不是孤岛。真实系统依赖 oracle、mempool、RPC、node、indexer 和数据库；基础设施错误同样会导致资金与账本事故。

本章题目

Q71 为什么智能合约不能直接请求普通 HTTP API?
Q72 一个可靠 Oracle 应具备哪些性质?
Q73 什么是 MEV?
Q74 Front-running、Back-running、Sandwich 在机制上如何区分?
Q75 为什么 Public Mempool 会产生独特安全问题?
Q76 JSON-RPC 在 Web3 架构中是什么?
Q77 Archive Node、Full Node、Light Client 有什么区别?
Q78 为什么 Web3 应用通常需要 Indexer?
Q79 Indexer 如何正确处理 Chain Reorg?
Q80 RPC Provider 挂了，生产 dApp 应如何降级?

Q71 为什么智能合约不能直接请求普通 HTTP API?

难度 ★★ | 高频度 ★★★★★ | 岗位 全栈/Oracle

面试官意图 从确定性共识解释 oracle 必要性。

30 秒回答

EVM 执行必须让所有验证节点对同一输入得到同一结果。普通 HTTP 响应可能因时间、地理、服务故障或个性化而不同，所以不能作为同步 EVM opcode 的非确定性输入。

深入解析

Oracle 的作用不是“让合约联网”，而是把链外事实通过某种可验证/可治理的数据发布机制转换成链上状态。这个过程必然引入新的信任与延迟。

也可以采用请求-响应异步 oracle：合约 emit/request，链下节点获取数据，再用另一笔 transaction 回写结果；关键是结果如何认证与聚合。

必须说出的关键点

- deterministic replicated execution
- HTTP is nondeterministic external world
- oracle posts data onchain
- asynchronous architecture

高频追问

- 为什么 RPC `eth_call` 可以访问互联网的前端数据？那是链外应用，不参与 consensus。
- 预言机能否证明天气是真的？取决于 data source/attestation model。
- TLSNotary/zkTLS 类技术试图解决什么？对 web response 提供更强可验证性。

常见误区 / Gotchas

- 说“因为 Solidity 没有 HTTP 库”。
- 把 oracle 当无信任事实源。

记忆锚点 链上不能随便问互联网，因为每个节点必须在没有外部不确定性的情况下重复得到同一结果。

Q72 一个可靠 Oracle 应具备哪些性质?

难度 ★★★ | 高频度 ★★★★★ | 岗位 DeFi/安全

面试官意图 将 oracle 从“价格 API”升级为风险系统。

30 秒回答

关键维度包括数据源多样性、聚合抗异常、更新新鲜度、staleness/deviation checks、市场流动性、故障 fallback、链上权限和明确的信任假设。

深入解析

价格 feed 可以“数值正确但已经过期”，也可以“更新及时但底层市场被操纵”。所以消费 oracle 的合约应同时检查 timestamp/round、合理区间和业务级风险。

L2 上还可能存在 sequencer downtime：如果 sequencer 刚恢复，价格与用户交易可见性可能异常，协议需要 grace period。

必须说出的关键点

- freshness
- deviation
- source diversity
- market liquidity
- fallback/circuit breaker
- sequencer-awareness on L2

高频追问

- Median 为什么常比 mean 更抗异常？极端 outlier 影响更小。
- oracle heartbeat 与 deviation trigger 如何共同工作？时间与价格变化双触发。
- fallback price 设为最后一次价格安全吗？长时间停更时风险增大。

常见误区 / Gotchas

- 只检查 `price > 0`。
- 不记录 feed decimals，造成数量级错误。

记忆锚点 Oracle 消费端至少问：值对不对、够不够新、来源是否健康、异常时我怎么办。

Q73 什么是 MEV?

难度 ★★★ | 高频度 ★★★★★ | 岗位 协议/DeFi/Infra

面试官意图 考交易排序经济学与 block-building 角色。

30 秒回答

MEV 指参与交易排序、包含或排除的主体能从这些排序权中提取的额外价值。常见来源包括跨市场套利、清算竞争和对公开交易意图的排序响应。

深入解析

现代 Ethereum 需要区分 searcher、builder、relay、proposer/validator 等角色。MEV 并非全部恶意：套利和清算可以提高市场效率，但有害 ordering 会让用户获得更差成交。

协议设计的目标通常不是“消灭 MEV”，而是减少用户可被提取的价值、明确 auction/order flow 规则，并避免排序权造成协议安全漏洞。

必须说出的关键点

- ordering/inclusion/exclusion value
- arbitrage/liquidation can be beneficial
- user-harming extraction exists
- PBS separates block building roles

高频追问

- 为什么 public mempool 让 order flow 有价值？交易意图在执行前可见。
- batch auction 如何改变 MEV？弱化连续时间排序优势。
- MEV 与普通交易手续费有什么区别？一个是额外经济价值，一个是资源费用。

常见误区 / Gotchas

- 把 MEV 只等同于单一攻击。
- 认为 private mempool 自动消除所有 MEV。

记忆锚点 MEV 的源头是：区块不是一个“无序集合”，谁决定顺序，谁就可能拥有经济权力。

Q74 Front-running、Back-running、Sandwich 在机制上如何区分?

难度 ★★★ | 高频度 ★★★★☆ | 岗位 DeFi/安全

面试官意图 测试 transaction ordering 基础，重点放在防御。

30 秒回答

Front-running 是交易被排在目标之前，back-running 是利用目标执行后的新状态紧随其后；sandwich 是在目标前后各放一笔交易，让目标在更不利状态执行。

深入解析

面试回答应立即转向防御：链上 `minOut/maxIn`、合理 deadline、limit order、batch auction、private order flow/commit-reveal 等都能减少特定排序风险。

不要依赖“前端显示的报价”作为安全约束，因为 block builder 只执行合约 calldata；真正的用户保护必须编码在交易参数和合约条件中。

必须说出的关键点

- ordering semantics
- onchain slippage bounds
- deadline/limit constraints
- protocol design can reduce ordering sensitivity

高频追问

- 为什么 `amountOutMin=0` 危险？交易接受任意差的价格。
- private submission 的信任是什么？依赖 relay/builder confidentiality policy。
- commit-reveal 的成本？两阶段 UX 和 liveness。

常见误区 / Gotchas

- 给出可执行的攻击步骤而不讲防御。
- 把所有 front-running 都等同于“黑客”。

记忆锚点 排序攻击防御原则：不要把“我期望的价格”留在前端，要把“我能接受的最差结果”写进链上约束。

Q75 为什么 Public Mempool 会产生独特安全问题?

难度 ★★★ | 高频度 ★★★★☆ | 岗位 后端/DeFi

面试官意图 理解“签名后、执行前”这一公开窗口。

30 秒回答

交易在进入公开 mempool 后，calldata、value、fee 和意图可能在被打包前就能被其他参与者看到和模拟。其他交易可以根据它调整自己的 fee/ordering，因此链上应用不能假设请求按提交顺序私密执行。

深入解析

这与传统 HTTPS API 很不同：请求到 server 通常不会被所有竞争者提前看到。Web3 合约应假设所有 pending intent 都可能公开，并用 slippage、nonce、commit-reveal 或 private flow 控制风险。

同时要考虑 replacement：用户可以用同 nonce 更高费用替换交易，后端不能只追踪 tx hash。

必须说出的关键点

- pre-execution visibility
- adversarial simulation
- ordering not FIFO
- same-nonce replacement

高频追问

- pending tx 是否一定公开？private relays/order flow 可能不公开。
- 为什么 secret 写进 calldata 没意义？进入链前后都可能被观察。
- 如何提交真正需要保密的信息？commit-reveal/ZK/encryption-based protocols。

常见误区 / Gotchas

- 把 mempool 当消息队列 FIFO。
- 后端只存 tx hash，不存 account+nonce 状态。

记忆锚点 Public mempool 意味着：你的交易不是“执行时才公开”，而是“别人可能先看懂，再决定怎么围绕它交易”。

Q76 JSON-RPC 在 Web3 架构中是什么?

难度 ★★ | 高频度 ★★★★★ | 岗位 后端/全栈

面试官意图 基础设施基础题。

30 秒回答

JSON-RPC 是应用与区块链节点交互的常见远程调用接口。前端/后端通过 provider 调用 `eth_call`、`eth_getLogs`、`eth_sendRawTransaction`、`eth_getBlockByNumber` 等方法读取、模拟或广播交易。

深入解析

RPC provider 不是区块链本身，而是你访问节点的入口。应用如果只依赖单一第三方 endpoint，就可能有 rate limit、错误数据、延迟、审查和单点故障。

`eth_call` 在指定 block state 上模拟消息调用，不产生链上交易；`eth_sendRawTransaction` 只广播已签名 bytes；`eth_getLogs` 适合按 block/topic 查询 logs，但大范围查询常被 provider 限制。

必须说出的关键点

- RPC != consensus
- read/simulate/broadcast methods
- block tag matters
- provider redundancy

高频追问

- `latest` 与 `finalized` block tag 有何区别？确认层级。
- RPC 返回 tx hash 是否表示执行成功？不表示。
- 为什么 archive query 可能失败？节点未保留历史 state。

常见误区 / Gotchas

- 把 Infura/Alchemy 之类 provider 当“链”。
- 所有读取都使用 latest，造成一致性 race。

记忆锚点 RPC 是“访问节点的 API”，节点才是执行/验证链状态的实体。

Q77 Archive Node、Full Node、Light Client 有什么区别?

难度 ★★★ | 高频度 ★★★★☆ | 岗位 Infra/后端

面试官意图 历史状态与验证成本题。

30 秒回答

Full node 验证链并维护当前运行所需状态；archive node 额外保存大量历史 state versions，支持查询过去任意 block 的账户/storage；light client 不保存/执行全部数据，而利用 headers、committee/proofs 等验证必要信息。

深入解析

“Full node 有全部历史区块”与“能查询任意历史 state”不是一回事。很多普通节点会保留历史 block 数据但 prune 旧 state trie，因此过去 block 的 `eth_call` 需要 archive capability。

Light client 的目标是让资源受限设备在不信任单个 RPC 的情况下验证关键链头/证明。

必须说出的关键点

- current state vs historical state
- archive is storage-heavy
- light client verifies proofs/headers
- query requirements drive node type

高频追问

- 做 indexer 一定需要 archive node 吗？从 genesis 自己顺序处理不一定；任意历史 state lookup 可能需要。
- trace API 为什么更重？重放/保存 execution traces。
- 节点 pruning 对 debug 有何影响？旧 state unavailable。

常见误区 / Gotchas

- 把 archive node 当“更安全的 full node”。
- 认为 light client 等于“信任第三方 API”。

记忆锚点 Full node 重点是验证现在；Archive 重点是还能回答“过去那个 block 当时的完整状态是什么”。

Q78 为什么 Web3 应用通常需要 Indexer?

难度 ★★★ | 高频度 ★★★★★ | 岗位 后端/全栈

面试官意图 链上状态与产品查询层的边界。

30 秒回答

区块链适合共识和可验证状态，不适合产品常见的复杂历史查询。Indexer 顺序消费 blocks/logs/transactions，把链上事件解析、规范化并存进数据库，为 UI、搜索、统计、通知提供低延迟查询。

深入解析

典型问题“某用户过去 90 天所有 swap 按 token 汇总”如果每次实时扫链会非常昂贵。Indexer 把昂贵的顺序扫描变成一次 ingestion，之后由数据库建立索引。

正确性关键是 reorg-aware：派生数据库只是 canonical chain 的 materialized view，链回滚时必须撤销对应派生 rows。

必须说出的关键点

- materialized offchain view
- logs -> normalized DB
- idempotent ingestion
- reorg rollback

高频追问

- 为什么不能把 indexer DB 当 source of truth？最终来源仍是链。
- event schema 升级怎么处理？按 contract version/implementation/block range decode。
- 如何 backfill 新字段？重新扫历史 raw events。

常见误区 / Gotchas

- 消费 event 后直接 append，无法 rollback。
- 只保存解析后数据，不保存 block/tx/log identity。

记忆锚点 Indexer 是链的“可回滚物化视图”，不是另一份永远正确的主账本。

Q79 Indexer 如何正确处理 Chain Reorg?

难度 ★★★★ | 高频度 ★★★★★ | 岗位 后端/交易所

面试官意图 真实生产事故题。

30 秒回答

每处理一个 block 都保存 number、hash、parentHash 以及该 block 对数据库造成的可逆变更。新 block 的 parentHash 与本地 canonical tip 不一致时，寻找 common ancestor，按逆序回滚 orphaned blocks，再应用新 canonical branch。

深入解析

数据表最好带 `block_number/block_hash/tx_hash/log_index` 唯一身份，写入要幂等。对于余额/聚合值，可使用 append-only event + recompute，或记录 undo log/transactional delta。

还应区分 `seen/pending/confirmed/finalized` 业务状态。高风险业务可以只在 finalized 后对用户产生不可逆外部副作用，或在提前 credit 时保留补偿机制。

必须说出的关键点

- parentHash continuity
- find common ancestor
- rollback orphan branch
- idempotent replay
- confirmation/finality states

高频追问

- 如果 reorg 深度超过本地缓存怎么办？从持久 checkpoint 往回找。
- Webhook 已经发给外部系统怎么撤销？需要补偿事件而不是幻想 exactly-once。
- 数据库怎样保证 rollback 原子？每 block transaction/delta log。

常见误区 / Gotchas

- 用 block number 作为唯一键，忽略同高度不同 hash。
- reorg 后只“覆盖新数据”但没删除旧派生状态。

记忆锚点 Reorg 处理的本质：数据库必须能把最近一段链“倒带”，再沿新分支重放。

Q80 RPC Provider 挂了，生产 dApp 应如何降级?

难度 ★★★ | 高频度 ★★★★★ | 岗位 全栈/Infra

面试官意图 可靠性与去中心化实际落地题。

30 秒回答

至少准备多 provider / 自建节点冗余、timeout、retry with jitter、circuit breaker、health scoring 和请求类型路由。关键读取可向多个节点交叉验证，写交易可安全地重复广播同一 signed raw transaction。

深入解析

读请求要注意 block consistency：如果 RPC A 的 latest=100、B=99，简单随机负载均衡可能让 UI 倒退。可固定 block tag、sticky session 或对 chain head 做一致性层。

写路径要区分“请求超时”和“交易未提交”。如果 raw tx 已经到节点但 HTTP response 丢失，再广播同 bytes 是幂等的；重新签一个新 nonce 则可能制造双操作。

必须说出的关键点

- multi-provider
- timeout/retry/circuit-breaker
- block-height consistency
- raw-tx rebroadcast is safer than blind re-sign

高频追问

- 如何检测 provider 返回错误 chain？检查 chainId/genesis/known contracts。
- 多 RPC disagreement 怎么处理？按 finalized hash 或 quorum。
- 第三方 RPC 限流时如何 backpressure？queue + caching + rate control。

常见误区 / Gotchas

- “区块链去中心化，所以单 RPC 挂了没问题”。
- HTTP timeout 后直接生成第二笔业务交易。

记忆锚点 去中心化协议不等于你的应用基础设施自动高可用；RPC 仍然要按传统分布式系统标准设计。

CHAPTER 09

Solana 与 Multi-chain：另一种执行模型

本章目标 不要把 Solana 当“更快的 EVM”。它的 program/account 分离、显式账户列表、PDA/CPI 和并行执行模型带来完全不同的工程边界。

本章题目

Q81 Ethereum 与 Solana 最大的编程模型区别是什么?
Q82 Solana Account 是什么?
Q83 为什么 Solana Program 通常被称为 Stateless?
Q84 什么是 PDA? 为什么没有私钥?
Q85 什么是 CPI?
Q86 Solana 为什么更容易并行执行某些 Transactions?
Q87 Ethereum 与 Solana Transaction Atomicity 有何异同?
Q88 为什么 Multi-chain dApp 比单链难很多?
Q89 Multi-chain Token 如何保证 Total Supply Invariant?
Q90 设计一个 Ethereum + Solana Wallet SDK。

Q81 Ethereum 与 Solana 最大的编程模型区别是什么?

难度 ★★★ | 高频度 ★★★★★ | 岗位 Multi-chain/Solana

面试官意图 确认候选人不会把不同链强行套成同一套“contract storage”心智。

30 秒回答

Ethereum 合约地址通常同时承载 code 与其持久 storage；Solana program 主要承载可执行代码，所有可变状态放在独立 accounts 中，并由 instruction 显式传入需要读取/写入的 accounts。

深入解析

这种设计让 runtime 在执行前知道 transaction 的 account access set，因此可以识别不冲突的事务并行调度。代价是开发者必须明确 account ownership、writable/signer 权限、PDA seeds 和 account validation。

Solana 官方 core docs 明确将 Accounts、Programs、Instructions、Transactions、PDAs、CPIs 作为基础概念，并说明 programs 是 stateless、mutable state 位于 data accounts。

必须说出的关键点

- Ethereum: contract-centric state
- Solana: program code + separate data accounts
- explicit account access
- parallelism from conflict information

高频追问

- Solana program 真正“无状态”是什么意思？代码本身不保存可变应用状态。
- 为什么账户列表错误会成为安全漏洞？恶意用户可传入错误 account。
- EVM 能否并行？理论上可做访问分析，但默认 transaction 语义更共享/顺序。

常见误区 / Gotchas

- 把 Solana account 等同于 EOA。
- 认为 program 无状态意味着应用没有状态。

记忆锚点 Ethereum 更像“对象自己带状态”；Solana 更像“函数程序 + 显式传入的一组状态对象”。

参考：R12 Solana Core

Q82 Solana Account 是什么?

难度 ★★ | 高频度 ★★★★★ | 岗位 Solana

面试官意图 最基础但最容易被 EVM 经验误导。

30 秒回答

Solana account 是以 32-byte address 标识的状态容器，包含 lamports、data、owner、executable 等字段。它可以是普通资金账户、程序数据账户或 executable program account，概念远宽于 Ethereum EOA。

深入解析

`owner` 表示哪个 program 有权修改该 account 的 data；并不是“这个用户拥有这个账户”的自然语言 owner。用户签名决定 transaction authorization，而 program ownership 决定 data mutation 权限。

程序处理 instruction 时必须验证传入 account 的 owner、address/PDA、signer、writable、data schema 等，否则会出现 arbitrary account injection。

必须说出的关键点

- addressed state object
- owner = program allowed to mutate data
- signer/writable are tx privileges
- account validation is core security

高频追问

- System Program owner 的普通账户意味着什么?
- executable account 与 data account 如何关联？依具体 loader/program model。
- 为什么不能只看 account data 就信任其类型？要检查 owner/discriminator。

常见误区 / Gotchas

- 把 owner 字段解释成“用户钱包地址”。
- 忘记验证传入 token mint/token account 的关系。

记忆锚点 Solana Account 首先是“状态容器”；它是不是钱包、程序、vault，要看 owner/data/executable 语义。

参考：R12 Solana Core

Q83 为什么 Solana Program 通常被称为 Stateless?

难度 ★★ | 高频度 ★★★★☆ | 岗位 Solana

面试官意图 进一步确认 code/data 分离。

30 秒回答

Program 自身负责执行逻辑，但应用的可变状态存放在外部 data accounts 中；调用者通过 instruction 把这些 accounts 传入。于是同一 program code 可以作用于大量不同状态对象。

深入解析

这类似“纯代码模块 + 显式数据库记录”，但 program 执行仍然能修改被允许 writable 且由正确 program 所有的 accounts，所以整个应用当然有状态。

这种模型也使升级 program code 与迁移 data account schema 成为两个不同问题；升级不能自动修改所有已有 data layout。

必须说出的关键点

- program code separated from mutable data
- state passed explicitly
- one program manages many accounts
- schema migration remains engineering concern

高频追问

- Program upgrade authority 是什么风险？可替换 code。
- Anchor account discriminator 有什么作用？帮助类型识别，不替代全部权限验证。
- 一个 program 能修改别的 program-owned account 吗？通常不能，除非通过 CPI 让 owner program 执行。

常见误区 / Gotchas

- 把 stateless 当作“不会写链上状态”。
- 忽略 upgrade authority。

记忆锚点 Solana 的“stateless program”说的是代码与数据分离，不是应用没有状态。

参考：R12 Solana Core

Q84 什么是 PDA？为什么没有私钥？

难度 ★★★ | 高频度 ★★★★★ | 岗位 Solana/钱包

面试官意图 Solana 账户寻址与 program authority 核心。

30 秒回答

Program Derived Address 是由 program id 与 seeds 确定性派生的 32-byte 地址，并被选在 Ed25519 curve 之外，因此不存在正常私钥。对应 program 可在 CPI 中通过 `invoke_signed` 和正确 seeds 让 runtime 认可 PDA signer authority。

深入解析

PDA 常用于 vault、user position、config、escrow 等 program-controlled state。Deterministic address 让客户端不用中央数据库就能根据 seeds 找到状态。

安全关键是 seed namespace：如果 seeds 不包含用户/market/类型等必要域，可能出现地址碰撞或不同业务共享错误 PDA。还要验证 bump 和 expected address。

必须说出的关键点

- deterministic seeds + program id
- off-curve, no private key
- program signs via runtime/invoke_signed
- namespace design matters

高频追问

- canonical bump 是什么？尝试 bump 直到派生地址 off-curve。
- PDA 是否天然是 account？地址可先派生，account 仍需创建并分配空间/owner。
- 攻击者能否自己签 PDA？不能持有私钥，必须经 owning program logic。

常见误区 / Gotchas

- 说 PDA 是“程序的私钥地址”。
- 只验证 seeds 中一部分导致错误授权。

记忆锚点 PDA 是“程序可证明控制的确定性地址”，控制权来自 runtime + seeds，而不是私钥。

参考：R13 Solana PDA

Q85 什么是 CPI?

难度 ★★★ | 高频度 ★★★★★ | 岗位 Solana/DeFi

面试官意图 Solana composability 与权限传递题。

30 秒回答

Cross-Program Invocation 是一个 Solana program 在执行过程中调用另一个 program instruction。它类似合约间调用，但账户、signer、writable privileges 都显式传递，并受 runtime privilege rules 限制。

深入解析

`invoke` 用现有 signer 权限调用；`invoke_signed` 允许当前 program 用 PDA seeds 让 runtime 赋予 PDA signer。CPI 是 token transfer、DEX composition、vault 策略等的基础。

安全上要注意 callee 可能执行复杂逻辑，调用前后的 account 状态与 reentrancy/privilege 假设都应审计。Solana 官方文档强调 CPI privilege extension 和 PDA signing 规则。

必须说出的关键点

- program-to-program call
- explicit accounts
- invoke / invoke_signed
- signer/writable privilege propagation

高频追问

- CPI 能提升一个原本 readonly account 为 writable 吗？不能超出调用者已有权限。
- PDA signer 权限如何传给下游 token program？invoke_signed。
- CPI depth 是否无限？runtime 有调用深度限制。

常见误区 / Gotchas

- 把 CPI 当普通函数调用，忽略账户权限。
- 给任意 callee program id 而不校验，造成恶意 CPI。

记忆锚点 CPI = 带显式账户能力(capabilities)的跨程序调用。

参考：R14 Solana CPI

Q86 Solana 为什么更容易并行执行某些 Transactions?

难度 ★★★★ | 高频度 ★★★★★ | 岗位 Solana/系统

面试官意图 从显式 account access set 推导运行时调度。

30 秒回答

Transaction message 会声明涉及的 accounts，并标识 signer/writable。Runtime 因此能提前知道读写集合：如果两笔 transaction 不争用同一 writable state，就可以并行执行，而不必假设它们任意访问全局状态。

深入解析

可以用数据库冲突图理解：write-write 与 write-read 冲突需要序列化，互不冲突的 account sets 可以并行。这也解释了“热门单一 account”会成为 contention hotspot。

所以 Solana 性能优化不仅是减少 compute units，还包括把状态设计得可分片，避免所有用户都写同一个 global account。

必须说出的关键点

- explicit read/write sets
- conflict-based scheduler
- hot account limits parallelism
- state sharding is app-level performance design

高频追问

- 一个 global counter 为什么会拖慢并发？所有 tx 写同一 account。
- 如何设计 per-user PDA 降低冲突？把独立状态拆分。
- readonly shared account 会阻止并行吗？多个只读通常可并行。

常见误区 / Gotchas

- 认为 Solana 每笔 transaction 都自动并行。
- 只优化 compute，不优化 account contention。

记忆锚点 Solana 并行性的秘密不是“硬件更快”，而是交易提前声明了自己要碰哪些状态。

Q87 Ethereum 与 Solana Transaction Atomicity 有何异同?

难度 ★★★ | 高频度 ★★★★☆ | 岗位 Multi-chain

面试官意图 跨链执行语义基础。

30 秒回答

两者都把一笔 transaction 视为原子状态转换：内部包含的多步操作若最终失败，相关状态不会部分提交。Solana transaction 可包含多条 instructions；Ethereum transaction 可触发多层 contract calls。

深入解析

区别在资源和状态声明：Solana 交易 message 显式列出 account metas 和 recent blockhash，并受 transaction size/compute unit 等约束；Ethereum 交易由一个 to/calldata 开始，内部动态访问任意状态。

原子性并不等于跨链原子性。Ethereum tx 与 Solana tx 之间没有共享 rollback domain，跨链协议必须自己设计 two-phase-like 状态机、timeouts 和 compensations。

必须说出的关键点

- atomic within one chain transaction
- multi-instruction/call-tree rollback
- cross-chain is not atomic by default
- resource models differ

高频追问

- 外部 oracle/HTTP 是否参与原子回滚？链外系统不会跟着回滚。
- CPI 失败怎样影响 Solana transaction？未捕获失败通常使整笔 tx 失败。
- bridge destination 执行失败怎么办？需要 retry/message state。

常见误区 / Gotchas

- 把“跨链 swap”宣传语理解成底层原子 transaction。
- 忽略链外 webhook/数据库不会自动 rollback。

记忆锚点 单链 transaction 可以 all-or-nothing；跨链流程天然跨越多个独立的原子域。

Q88 为什么 Multi-chain dApp 比单链难很多?

难度 ★★★ | 高频度 ★★★★★ | 岗位 全栈/架构

面试官意图 考 abstraction 是否建立在真实差异之上。

30 秒回答

不同链在 finality、address/signature、fee、token standard、RPC、execution、nonce、transaction size 和 error model 上都可能不同。Multi-chain SDK 必须统一产品体验，同时不能把这些关键差异错误抹平。

深入解析

最危险的是 false abstraction：提供一个统一 `send()` API，却把 EVM nonce replacement、Solana blockhash expiry、不同 confirmation 模型等隐藏到无法正确恢复错误的程度。

好的抽象会统一 intent/result state machine，但保留 chain capability：例如 `supportsPermit`、`supportsAccountAbstraction`、`confirmationPolicy`、`nativeFeeModel`。

必须说出的关键点

- capability-based abstraction
- chain-specific finality
- different signer/address/fee models
- normalize errors without losing semantics

高频追问

- 统一 transaction status 应有哪些？created/signed/submitted/included/finalized/failed/replaced/expired。
- token decimals/metadata 如何安全处理？链上 authoritative + curated metadata。
- 如何测试多链 SDK？local validator/fork/testnet + deterministic fixtures。

常见误区 / Gotchas

- 用 `if chainId == ...` 到处散落分支。
- 把所有链都映射成 EVM nonce/gas 心智。

记忆锚点 Multi-chain 的目标不是“让所有链看起来一样”，而是“在统一接口下显式管理它们不一样的地方”。

Q89 Multi-chain Token 如何保证 Total Supply Invariant?

难度 ★★★★ | 高频度 ★★★★★ | 岗位 Bridge/Token/安全

面试官意图 跨链资产 accounting 核心。

30 秒回答

以 lock-and-mint 为例，source 链原生资产进入不可流通锁仓，destination 链铸出 wrapped supply。系统必须保证每个可流通 wrapped token 都有对应 backing，且 message 不可重放或重复执行。

深入解析

可以写全局不变量：`circulating_native + locked_native?` 要按模型明确定义；更实用的是 `wrapped_outstanding <= verifiably_locked_backing`。Burn-and-mint 模型则要求跨链总 mint/burn 守恒。

失败模式包括 source reorg、destination duplicate mint、validator forgery、unlock without burn、decimal conversion rounding、emergency admin overmint。

必须说出的关键点

- one message executes once
- backing vs wrapped supply
- reorg/finality before mint
- rate limits bound loss

高频追问

- 为什么 bridge 要等 source finality? 避免已 mint wrapped 后 source lock 被 reorg。
- decimal 不同如何处理? 明确 canonical units 和 rounding invariant。
- 链 B 停止时如何退出? 取决于 bridge proof/guardian/escape design。

常见误区 / Gotchas

- 只检查每条链自己的 totalSupply。
- 没有 global message nonce/id。

记忆锚点 跨链 token 的安全不是 `totalSupply()` 一个数字，而是所有链之间“锁、烧、铸、解锁”的守恒。

Q90 设计一个 Ethereum + Solana Wallet SDK。

难度 ★★★★★ | 高频度 ★★★★★ | 岗位 Wallet/Staff/Multi-chain

面试官意图 综合 SDK、签名、transaction lifecycle 与错误抽象题。

30 秒回答

建议用 chain adapter 架构：统一 account/session/intent API，EVM adapter 处理 nonce/EIP-1559/typed data/AA，Solana adapter 处理 recent blockhash/account metas/versioned transaction/fee payer；上层统一返回可观测 transaction state。

深入解析

Key management 层不要假设同一 key algorithm：Ethereum 常见 secp256k1，Solana 常见 Ed25519。Signer interface 应面向“sign message/sign transaction capability”，而不是暴露 raw private key。

Transaction builder、simulation、fee estimation、confirmation、token metadata、hardware wallet、RPC failover 都应模块化。失败状态必须保留链特定原因，例如 EVM replacement 与 Solana blockhash expired。

必须说出的关键点

- chain adapters
- signer capability interface
- transaction state machine
- simulation before send
- RPC redundancy
- hardware/session key support

高频追问

- 同一助记词能否派生两条链 keys？可以通过不同 derivation curves/path，但实现需明确。
- 如何防 dApp 请求错误链签名？domain/chain display + policy。
- 如何做 send retry？重广播 signed bytes vs rebuild expired transaction 取决于链。

常见误区 / Gotchas

- 一个 `privateKey: string` 贯穿所有模块。
- 把 chain-specific error 全部转换成 “Transaction failed”。

记忆锚点 跨链钱包 SDK 的最高层抽象是“用户意图和状态机”，不是“所有链都有同一种 transaction”。

CHAPTER 10

系统设计与现场实战

本章目标 高级 Web3 面试最终会回到软件工程：实现、审查、调试、可恢复性、幂等性、账本、一致性和安全边界。

本章题目

Q91 Coding：实现一个 Mini ERC-20 Staking Contract，怎样从 O(N) 奖励更新到 O(1)?
Q92 Coding Review：看到一个 Withdraw 函数，如何快速审查?
Q93 System Design：设计一个 NFT Marketplace。
Q94 System Design：设计 DAO Governance System。
Q95 System Design：设计 Multi-sig Wallet。
Q96 System Design：设计 Token Vesting Contract。
Q97 System Design：设计高可靠 Blockchain Indexing Service。
Q98 System Design：设计中心化交易平台 Deposit System。
Q99 Debug：用户说“交易失败了”，你如何系统排查?
Q100 Staff 题：从用户点击按钮到链上结算，设计完整 Web3 Protocol。

Q91 Coding：实现一个 Mini ERC-20 Staking Contract，怎样从 O(N) 奖励更新到 O(1)?

难度 ★★★★ | 高频度 ★★★★★ | 岗位 Solidity/DeFi

面试官意图 典型 live coding + accounting 题。

30 秒回答

不要为每次时间变化遍历所有 stakers。使用全局累计 `rewardPerTokenStored` 与每个用户的 `userRewardPerTokenPaid`、`rewards`，在用户 stake/withdraw/claim 时惰性结算差额，把全局更新降为 O(1)。

深入解析

核心是 index accounting：设总 stake 为 S，单位时间 reward rate 为 R，则一段时间内每个 staked token 累积 `R*dt/S` 的奖励指数。用户 earned = balance * (globalIndex-userIndex) + storedReward。

实现必须处理 S=0、precision scaling、reward period、token transfer safety、reentrancy 和 rounding。改变 reward rate 时先 checkpoint 旧 index，否则会把过去时间按新 rate 结算。

rewardPerToken += rewardRate * Δt * SCALE / totalStaked

earned(u)=bal(u)*(index-paid(u))/SCALE + stored(u)

必须说出的关键点

- global cumulative index
- lazy per-user checkpoint
- precision scaling
- update index before balance changes
- CEI / SafeERC20

高频追问

- 奖励 token 与 stake token 相同有什么额外 accounting 风险？合约余额混合。
- fee-on-transfer stake token 怎么办？按实际 received 记账。
- 如何 invariant fuzz？total staked、claim 不超 funding、index monotonic。

常见误区 / Gotchas

- 每秒 loop 全部用户。
- stake 后再 update reward，导致新资金吃到过去奖励。

记忆锚点 把“给 N 个用户发奖励”改写成“维护一个全局累计指数，每个用户只在交互时补差额”。

Q92 Coding Review：看到一个 Withdraw 函数，如何快速审查?

难度 ★★★ | 高频度 ★★★★★ | 岗位 Solidity/安全

面试官意图 公开面试很常见的 buggy-contract review 模式。

30 秒回答

按资金流顺序检查：谁能调用、amount 如何计算、余额何时更新、外部 call 在哪里、call 失败如何处理、token 是否非标准、是否可重入、event 是否反映最终状态、是否有每日/全局额度 invariant。

深入解析

第一眼先找“read balance -> external call -> write balance”这类时序错误。再看低级 `call` return value、unchecked token transfer、recipient 是否用户可控、是否存在 cross-function shared state。

最后构造恶意 receiver：fallback 回调、revert、耗 gas、调用另一个入口；如果合约涉及 ERC777/ERC721 hook，也把 callback 放入 threat model。

必须说出的关键点

- authorization
- state-before-interaction
- return-value handling
- token compatibility
- reentrancy/callback
- withdrawal invariant

高频追问

- `transfer` 是否绝对防重入？不应依赖 gas stipend 作为长期安全边界。
- push vs pull payment？pull 降低一个恶意 recipient 阻塞批处理。
- 如何写 regression test？恶意 receiver + state assertions。

常见误区 / Gotchas

- 只运行静态分析不读资金逻辑。
- 修复重入后忽略余额可以被 under/over-account 的业务 bug。

记忆锚点 Withdraw review 就问三件事：钱从哪算出来、账什么时候改、控制权什么时候交给别人。

参考：R5 Solidity Security

Q93 System Design：设计一个 NFT Marketplace。

难度 ★★★★ | 高频度 ★★★★★ | 岗位 Solidity/全栈

面试官意图 综合订单、授权、签名、结算和索引。

30 秒回答

先决定 listing 是 onchain custody/状态还是 offchain signed order。现代高效设计常让 seller 保留 NFT，通过 EIP-712 签订单，成交时 settlement contract 验签、nonce/expiry、ownership/approval，并原子转移 NFT 与资金。

深入解析

订单至少绑定 maker、asset contract、tokenId/amount、payment token、price、nonce、expiry、chain/domain。取消可用 per-order nonce、counter increment 或 bitmap。

结算要处理 royalties/fees、ERC721/1155、安全转账 callback、fee-on-transfer token、native ETH refund、partial fill（若支持）、reentrancy 和 signature replay。链下 order book/indexer 只是发现订单，链上 settlement 才是最终约束。

必须说出的关键点

- offchain order + onchain settlement
- EIP-712 domain/nonce/expiry
- approval/ownership checked at fill time
- atomic asset/payment transfer
- indexer is not source of truth

高频追问

- 为什么 listing 时不锁 NFT？更低 gas/custody，但订单可能变 stale。
- 订单价格变更如何取消旧单？nonce/counter。
- royalty 是否协议强制？取决于 marketplace policy/standard。

常见误区 / Gotchas

- 只设计 CRUD API，不设计链上重放/授权。
- 成交前不重新检查 NFT ownership/approval。

记忆锚点 Marketplace 的核心不是“展示 NFT”，而是把一个可撤销的卖方意图安全地变成原子结算。

Q94 System Design：设计 DAO Governance System。

难度 ★★★★ | 高频度 ★★★★★ | 岗位 Governance/Protocol

面试官意图 投票只是表面，真正考 proposal lifecycle 与治理攻击。

30 秒回答

标准流程可拆为 proposal -> voting delay -> snapshot voting power -> voting period -> quorum/threshold -> timelock queue -> execution。关键参数与权限也应由 governance 本身或受限 guardian 管理。

深入解析

使用 snapshot block 的原因是防止投票期间转移同一 token 重复投票，也降低临时借贷投票权攻击。Delegation 把 token ownership 与 voting participation 分离。

Timelock 给监控者观察执行内容的窗口。Emergency guardian 可以 pause，但不应拥有无限 treasury transfer 权。治理系统还要防 proposal spam、malicious calldata、quorum manipulation 和 governance capture。

必须说出的关键点

- snapshot voting power
- proposal threshold/quorum
- timelock execution
- delegation
- guardian scope
- governance attack model

高频追问

- flash loan voting 为什么历史 snapshot 能缓解？借来的 token 不能改变过去 block balance。
- 治理能升级自己吗？若可以，升级路径是最高权限。
- 低 turnout 如何处理？quorum 与 delegation trade-off。

常见误区 / Gotchas

- 投票结束立刻执行高权限操作。
- guardian 可以任意转 treasury，实际成为 owner。

记忆锚点 DAO 安全的关键不是“大家能投票”，而是“投票权怎么取快照、提案何时生效、用户有没有反应时间”。

Q95 System Design：设计 Multi-sig Wallet。

难度 ★★★★ | 高频度 ★★★★★ | 岗位 Wallet/Solidity

面试官意图 签名聚合、nonce 和 owner management 基础系统设计。

30 秒回答

定义 owners 集合、threshold、transaction payload、nonce 和 executed state。执行前验证至少 threshold 个不同有效 owner signatures，签名消息绑定 chain、wallet address、nonce 和完整 call data，成功后消费 nonce。

深入解析

签名排序/去重可降低 gas 和防 duplicate signer。Owner add/remove、threshold change 本身也应通过 multisig 提案执行，避免单独 admin 后门。

高级设计加入 module/guard、daily limit、recovery、hardware signer 和 simulation，但每个扩展都会增加攻击面。

必须说出的关键点

- threshold unique signatures
- domain + nonce replay protection
- owner management via same governance
- arbitrary call is extremely powerful

高频追问

- 2-of-3 与 3-of-5 如何选？安全、可用性、地理/设备独立性权衡。
- 签名可以链下收集吗？可以，最终一笔 tx 提交聚合签名。
- 一个 owner key 丢失怎么办？剩余 threshold 必须能更新 owners。

常见误区 / Gotchas

- 只计签名数量，不检查 signer 去重。
- 签名没绑定 wallet/chain，跨部署重放。

记忆锚点 Multi-sig 的安全公式不是“有 N 把钥匙”，而是“至少 T 个独立授权才能执行一个绑定上下文的唯一操作”。

Q96 System Design：设计 Token Vesting Contract。

难度 ★★★ | 高频度 ★★★★☆ | 岗位 Solidity

面试官意图 考时间函数、claim accounting 与边界。

30 秒回答

定义 allocation、start、cliff、duration、claimed。线性 vesting 中，cliff 前 vested=0，结束后 vested=allocation，中间按时间比例增长；可领取 `claimable = vested - claimed`。

深入解析

所有计算用整数，要明确 rounding 方向，并保证 `claimed <= vested <= allocation`。Claim 前更新 claimed 再转 token；如果有 revoke，必须定义已归属部分归 beneficiary、未归属部分归谁。

批量给很多 beneficiary 时不要用单次循环把所有人结算；用 per-user schedule 或 Merkle claim 降低部署/storage 成本。

$$\text{claimable}(t) = \text{vested}(t) - \text{claimed}$$

$$0 \le \text{claimed} \le \text{vested}(t) \le \text{allocation}$$

必须说出的关键点

- monotonic vested(t)
- claimed <= vested
- cliff/start/end boundaries
- CEI on claim
- avoid O(N) distributions

高频追问

- block.timestamp 能否被操纵？存在小范围 producer influence，不适合超精细时间随机性，但长期 vesting 通常可接受。
- 多个 grants 怎么存？schedule id / per-user array，注意 gas。
- 转让 beneficiary 是否允许？明确业务语义。

常见误区 / Gotchas

- duration=0 除零。
- claim 后再更新 claimed，留下重入窗口。

记忆锚点 Vesting 合约最重要的 invariant：时间只让 vested 单调增加，任何时刻已领取都不能超过已归属。

Q97 System Design：设计高可靠 Blockchain Indexing Service。

难度 ★★★★★ | 高频度 ★★★★★ | 岗位 Backend/Staff

面试官意图 典型 Web3 后端 Staff 题。

30 秒回答

架构建议：multi-RPC ingestion -> canonical chain tracker -> raw block/log durable store -> decoder/versioned ABI -> transactional processors -> normalized DB/search -> API。每个 block 是一个可回滚的处理单元。

深入解析

Ingestion 记录 raw block/hash/parent，先保证数据可重放；processor 使用 `(chainId, blockHash, txHash, logIndex, handlerVersion)` 做幂等。遇到 reorg 从 common ancestor 回滚派生状态。

大规模系统要支持 horizontal partition by chain/range、backfill、live tail、checkpoint、dead-letter、schema evolution、ABI/proxy implementation versioning、rate limit 和 observability。

必须说出的关键点

- raw immutable ingestion
- canonical tracker
- idempotent processors
- reorg undo/replay
- versioned decoding
- backfill + live convergence

高频追问

- 如何保证 backfill 与 live 不重复？统一 event identity/idempotency key。
- proxy upgrade ABI 如何选择？按 implementation change block 建版本。
- API 读 finalized 还是 latest？按业务 consistency tier。

常见误区 / Gotchas

- 只保存最终业务表，原始 logs 丢失导致无法重放。
- 消费队列“exactly once”口号替代数据库幂等。

记忆锚点 专业 Indexer 的核心能力不是“扫得快”，而是“任何时候都能从原始链数据确定性重建正确派生状态”。

Q98 System Design：设计中心化交易平台 Deposit System。

难度 ★★★★★ | 高频度 ★★★★★ | 岗位 Backend/Exchange

面试官意图 金融账本、一致性和链重组综合题。

30 秒回答

流程：地址归属/标签 -> chain ingestion -> detect inbound transfer -> confirmation/finality policy -> deposit record -> double-entry ledger credit -> reconciliation。链上事件与内部账本之间必须通过幂等状态机连接。

深入解析

绝不能只做 `user.balance += amount`。内部账本要有不可变 journal：例如借记平台 onchain asset/clearing account，贷记用户 liability account，使每个 entry 借贷平衡并可审计。

风险包括 reorg/double credit、token decimals、fee-on-transfer、wrong chain、memo/tag、contract deposits、duplicate events、address reuse、hot/cold sweep。大额可采用更高 confirmation tier。

必须说出的关键点

- double-entry ledger
- idempotent deposit identity
- confirmation/finality tiers
- reorg compensation
- reconciliation with onchain balances

高频追问

- 为什么链上 balance != 用户 liabilities？还有 fees、hot/cold transfers、pending deposits。
- 重复看到同 tx/log 怎么办？唯一键 + 幂等。
- 已 credit 后发生深 reorg？冻结/逆向 ledger entry/风险储备，不能静默改历史。

常见误区 / Gotchas

- 把数据库 balance 当账本。
- 链上 watcher 和用户 credit 写在两个无法恢复的独立步骤。

记忆锚点 交易所入金系统的本质是把“可能回滚的外部链事件”安全映射成“可审计的内部金融账本”。

Q99 Debug：用户说“交易失败了”，你如何系统排查?

难度 ★★★★ | 高频度 ★★★★★ | 岗位 全栈/Support/Senior

面试官意图 非常实战，考故障树和层次化定位。

30 秒回答

先判断失败在哪一层：是否构造成功、签名、RPC 接收、mempool、inclusion、EVM execution、post-confirmation UI/indexer。拿到 chainId、from、nonce、tx hash/raw tx、receipt 和 revert data 后逐层缩小。

深入解析

如果没有 tx hash，检查钱包拒签、RPC preflight、余额/fee；有 hash 无 receipt，检查 nonce gap、underpriced replacement、provider mempool；有 receipt status=0，做 `eth_call`/trace 解码 custom error。

DeFi 还要查 allowance、balance、deadline、minOut、oracle/paused state；proxy 系统查 implementation/ABI version；后端展示错误则查 indexer reorg/lag。

必须说出的关键点

- classify lifecycle stage first
- nonce + fee + chainId
- receipt/revert/trace
- simulate same block state if possible
- separate chain failure from UI/indexer failure

高频追问

- 为什么当前 `eth_call` 可能无法复现过去 revert？state 已变化，要指定历史 block（可能需 archive）。
- tx pending 很久如何判断被 dropped/replaced？按 from+nonce 查。
- custom error 如何 decode？用 ABI selector。

常见误区 / Gotchas

- 第一反应“MetaMask bug”。
- 只看 tx hash explorer 页面，不看 raw input/revert。

记忆锚点 排障顺序：有没有发出去 -> 有没有进块 -> EVM 为什么回滚 -> 链上成功后为什么产品还显示失败。

Q100 Staff 题：从用户点击按钮到链上结算，设计完整 Web3 Protocol。

难度 ★★★★★ | 高频度 ★★★★★ | 岗位 Staff/Protocol/全栈

面试官意图 整本书的综合题：协议、安全、钱包、基础设施、数据与运维。

30 秒回答

先从用户 intent 出发，画出 Frontend -> Wallet/Smart Account -> signing -> RPC/bundler -> mempool/sequencer -> contracts -> oracle/bridge -> settlement -> events -> indexer -> DB/API -> UI 的端到端数据流，再画一张独立 trust-boundary 图。

深入解析

对每层写 failure mode：钱包签错 domain、RPC timeout、交易 replacement、sequencer censorship、oracle stale、contract paused、L1 reorg、indexer lag。然后设计 retry、idempotency、fallback、permissionless exit 与 observability。

资产安全层列出 invariants 和 privileged roles：谁能 upgrade、pause、改 oracle、提 treasury、替换 bridge verifier？关键权限应有 multisig/timelock/monitoring，并明确最坏损失。

最后定义 user-visible state machine：draft -> awaiting_signature -> submitted -> pending -> included -> confirmed/finalized；失败还要区分 rejected/reverted/replaced/expired。产品层不能把所有状态压成一个 spinner。

必须说出的关键点

- intent and transaction state machine
- trust-boundary diagram
- asset invariants
- privileged-role inventory
- RPC/indexer redundancy
- reorg/finality
- permissionless exit
- observability/runbooks

高频追问

- 如果 RPC 全挂怎么办？切换 provider/自建节点，用户签名资产不应被托管。
- 如果 admin key 被盗最大损失？取决于 role scope/timelock/rate limit。
- 如果 oracle 停更怎么办？circuit breaker/withdraw-only/fallback。
- 如果 L2 sequencer 停止怎么办？forced inclusion/escape path。

常见误区 / Gotchas

- 只画技术组件，没有状态机和信任边界。
- 声称“去中心化”却保留单 EOA upgrade key。

记忆锚点 Staff 级 Web3 设计的四张图：数据流、状态机、资产不变量、权限/信任边界。

APPENDIX

面试冲刺与速查

A. 最优先掌握的 30 题

01. Q11 EVM 是什么?
02. Q12 Stack / Memory / Storage / Transient Storage 有什么区别?
03. Q13 Solidity 的 storage、memory、calldata 有什么区别?
04. Q14 CALL 与 DELEGATECALL 有什么区别?
05. Q16 Mapping 在 EVM Storage 中如何定位?
06. Q18 EIP-1559 的费用机制如何工作?
07. Q21 public / external / internal / private 有什么区别?
08. Q23 receive() 与 fallback() 有什么区别?
09. Q30 如何系统做 Solidity Gas Optimization?
10. Q31 什么是 Reentrancy?
11. Q32 Checks-Effects-Interactions 为什么有效?
12. Q33 ReentrancyGuard 是否解决所有 Reentrancy?
13. Q34 为什么不能使用 tx.origin 做 Authorization?
14. Q35 Oracle Manipulation 的根因是什么?
15. Q36 Flash Loan 为什么不是漏洞本身?
16. Q37 Access Control 应如何设计?
17. Q38 Transparent Proxy 与 UUPS Proxy 有什么区别?
18. Q39 Upgradeable Contract 为什么不能随便改变 State Variable 顺序?
19. Q40 给你一份 200 行 Solidity 合约，如何系统做 Security Review?
20. Q41 Uniswap V2 为什么使用 x*y=k?
21. Q44 Uniswap V3 Concentrated Liquidity 解决了什么问题?
22. Q46 Lending Protocol 的 Health Factor 是什么?
23. Q52 Optimistic Rollup 的工作原理是什么?
24. Q53 ZK Rollup 与 Optimistic Rollup 的核心区别?
25. Q54 EIP-4844 为什么对 Rollup 重要?
26. Q55 Data Availability 到底是什么意思?
27. Q63 什么是 Signature Replay Attack?
28. Q67 ERC-4337 的 UserOperation 与 EntryPoint 流程是什么?
29. Q69 EIP-7702 与 ERC-4337 是什么关系?
30. Q97 System Design：设计高可靠 Blockchain Indexing Service。

B. 7 天冲刺计划

时间	范围	训练目标
Day 1	Q01-Q20	Blockchain + EVM；重点手推 storage / calldata / tx lifecycle
Day 2	Q21-Q40	Solidity + Security；把 CEI、Proxy、storage layout 讲到能画状态机
Day 3	Q41-Q60	DeFi + L2；手推 xy=k、HF、DA、Rollup trust model
Day 4	Q61-Q80	Wallet/AA + Infra；重点 712/4337/7702、reorg/RPC

Day 5	Q81-Q90	Solana/Multi-chain；重点 account/program/PDA/CPI 与冲突并行
Day 6	Q91-Q100	Coding + System Design；每题限时 30-45 分钟口述/画图
Day 7	五星题复盘	只看记忆锚点，自测追问，补错题和项目经历映射

C. Solidity Security 速查清单

- 资产：合约持有哪些 ETH/token/share/claim？最大 TVL/单笔价值是多少？
- 权限：owner/admin/upgrader/pauser/oracle setter/treasury 各自能造成什么最坏后果？
- 外部调用：CALL/delegatecall/token callback/hook 之前是否恢复 invariant？
- Accounting：shares/assets、debt/collateral、fee、rounding direction 是否始终一致？
- Oracle：freshness、deviation、decimals、liquidity、sequencer downtime、fallback？
- Upgrade：initializer、storage layout、implementation lock、authorizeUpgrade、timelock？
- DoS：unbounded loop、push payment、单个恶意用户能否阻塞全局进度？
- Token compatibility：fee-on-transfer、rebasing、ERC777 callback、non-standard return？
- Testing：unit + fuzz + invariant + fork + static analyzer + adversarial receiver。

D. Web3 System Design 速查框架

1. 先定义用户 Intent 与资产安全目标。
2. 画端到端数据流：Wallet -> RPC/Bundler -> Chain -> Logs -> Indexer -> DB -> UI。
3. 画交易状态机：Created -> Signed -> Submitted -> Included -> Finalized；外加 Replaced/Reorged/Failed。
4. 列出每个外部依赖：RPC、Oracle、Sequencer、Bridge、Signer、DB。
5. 为每个依赖写 Timeout / Retry / Idempotency / Fallback。
6. 列出 privileged roles 与最大 blast radius。
7. 定义资产/账本 invariants。
8. 设计 Reorg / replay / duplicate / backfill / disaster recovery。
9. 最后才谈缓存、分片、吞吐与成本优化。

参考资料（官方 / 一手资料）

[R1] Ethereum.org. Ethereum Virtual Machine (EVM). https://ethereum.org/developers/docs/evm/

[R2] Ethereum Improvement Proposals. EIP-4844: Shard Blob Transactions. https://eips.ethereum.org/EIPS/eip-4844

[R3] Ethereum Improvement Proposals. EIP-7702: Set Code for EOAs. https://eips.ethereum.org/EIPS/eip-7702

[R4] Ethereum Improvement Proposals. EIP-1153: Transient storage opcodes. https://eips.ethereum.org/EIPS/eip-1153

[R5] Solidity Documentation. Security Considerations. https://docs.soliditylang.org/en/latest/security-considerations.html

[R6] OpenZeppelin Contracts. Access Control. https://docs.openzeppelin.com/contracts/5.x/access-control

[R7] OpenZeppelin Contracts. Proxy - Transparent vs UUPS. https://docs.openzeppelin.com/contracts/5.x/api/proxy

[R8] OpenZeppelin Upgrades Plugins. Frequently Asked Questions - storage compatibility.
https://docs.openzeppelin.com/upgrades-plugins/faq

[R9] Uniswap. Uniswap v3 Core Whitepaper. https://app.uniswap.org/whitepaper-v3.pdf

[R10] Aave. Health Factor & Liquidations. https://aave.com/help/borrowing/liquidations

[R11] Ethereum Improvement Proposals. ERC-4337: Account Abstraction Using Alt Mempool.
https://eips.ethereum.org/EIPS/eip-4337

[R12] Solana Documentation. Core Concepts. https://solana.com/docs/core

[R13] Solana Documentation. Program Derived Addresses (PDAs). https://solana.com/docs/core/pda

[R14] Solana Documentation. Cross Program Invocation (CPI). https://solana.com/docs/core/cpi

[R15] Ethereum Improvement Proposals. EIP-1559: Fee market change for ETH 1.0 chain.
https://eips.ethereum.org/EIPS/eip-1559

注：协议规范与文档会持续更新。面试前应再次核验目标岗位所使用的 chain、Solidity/compiler、OpenZeppelin 和 protocol 版本。

END

真正的高级 Web3 面试答案：定义清楚、原理能推、边界敢讲、不变量能写、系统能落地。